

VOYAGER: 一个基于大语言模型的开放式具身智能体

Guanzhi Wang^{1,2✉}, Yuqi Xie³, Yunfan Jiang^{4*}, Ajay Mandlekar^{1*},
肖超伟 朱玉可^{1,5} 范麟熙 (Jim)^{1,3✉} 阿尼玛·阿南德库马尔^{1,2†}
1 英伟达, 2 加州理工学院, 3 德克萨斯大学奥斯汀分校, 4 斯坦福大学, 5 威斯康星大学麦迪逊分校 * 同等贡献 † 同等指导 #
通讯作者 <https://voyager.minedojo.org>

摘要

我们提出了 VOYAGER，这是首个由大语言模型驱动的具身终身学习智能体，能够在《我的世界》中持续探索世界、获取多样技能，并在无人干预的情况下做出新发现。VOYAGER 包含三个关键组件：1) 一个最大化探索的自动课程，2) 一个不断增长的可执行代码技能库，用于存储和检索复杂行为，以及 3) 一种新的迭代提示机制，该机制结合环境反馈、执行错误和自我验证来改进程序。VOYAGER 通过黑盒查询与 GPT-4 交互，从而无需微调模型参数。VOYAGER 开发的技能具有时间扩展性、可解释性和可组合性，这使智能体的能力迅速复合增长，并缓解了灾难性遗忘。实验表明，VOYAGER 展现出强大的上下文终身学习能力，并在游玩《我的世界》时表现出卓越的熟练度。与之前的 SOTA 相比，它获得的独特物品多 3.3 $\times$ ，旅行距离长 2.3 $\times$ ，解锁关键科技树里程碑的速度快 15.3 $\times$ 。VOYAGER 能够在一个新的《我的世界》世界中利用已学习的技能库从零开始解决新任务，而其他技术则难以泛化。

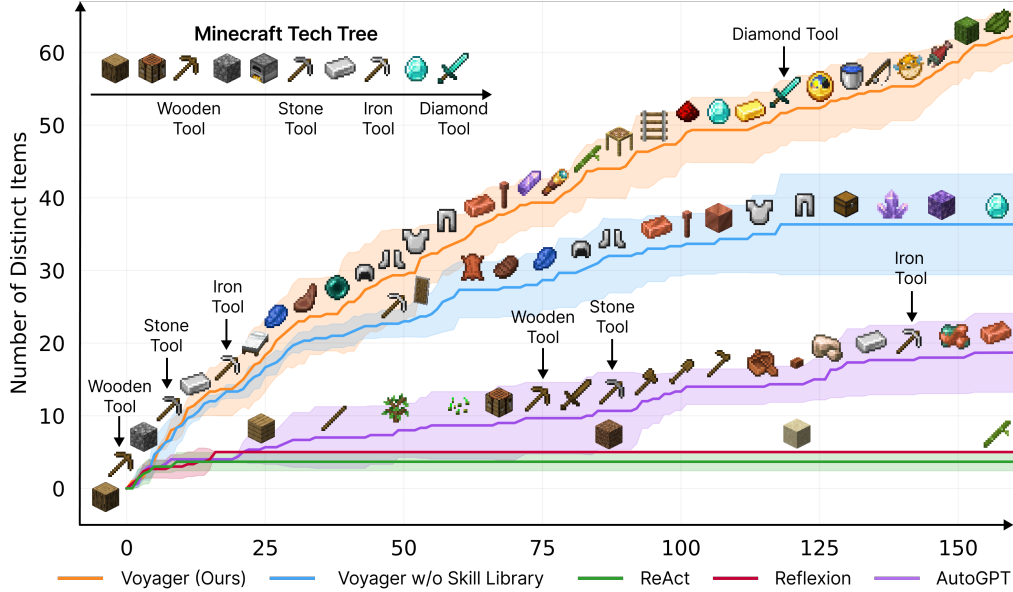


图 1: VOYAGER 通过自我驱动的探索持续发现新的《我的世界》物品和技能，显著优于基线。X 轴表示提示迭代次数。

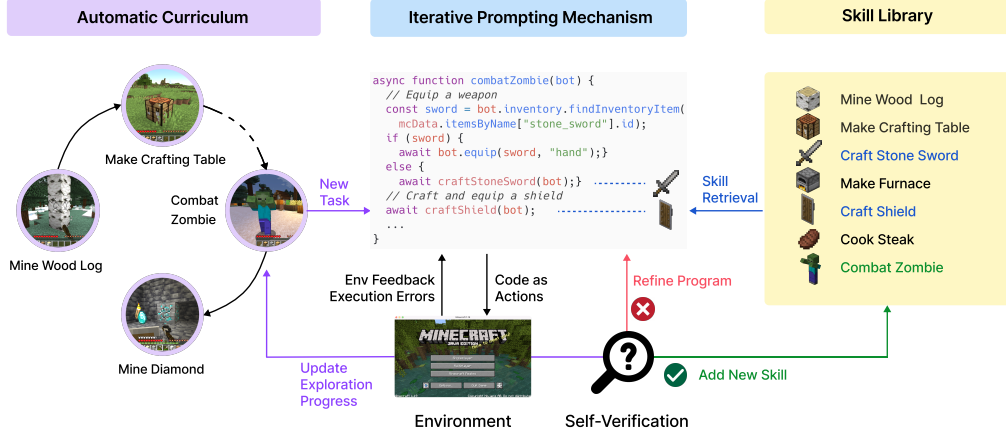


图 2: VOYAGER 由三个关键组件组成: 用于开放式探索的自动课程、用于日益复杂行为的技能库, 以及使用代码作为动作空间的迭代提示机制。

1 引言

构建能够在开放式世界中持续探索、规划并发展新技能的通用具身智能体, 是人工智能领域的重大挑战 [1 – 5]。经典方法采用在原始动作上运行的强化学习 (Reinforcement Learning) [6, 7] 和模仿学习 (Imitation Learning) [8 – 10], 这在系统探索 [11 – 15]、可解释性 [16 – 18] 和泛化 [19 – 21] 方面可能具有挑战性。基于大语言模型 (Large Language Model) 的智能体的最新进展利用预训练大语言模型中封装的世界知识来生成一致的动作计划或可执行策略 [16, 22, 19]。它们被应用于游戏和机器人等具身任务 [23 – 27], 以及无具身的自然语言处理任务 [28 – 30]。然而, 这些智能体并非终身学习者, 无法在长时间跨度内逐步获取、更新、积累和迁移知识 [31, 32]。

以《我的世界》为例。与人工智能领域研究的大多数其他游戏不同 [33, 34, 10], 《我的世界》没有预设的最终目标或固定故事情节, 而是提供了一个充满无限可能的独特游乐场 [23]。《我的世界》要求玩家探索广阔的程序化生成 3D 地形, 并利用收集的资源解锁科技树。人类玩家通常从学习基础知识开始, 例如挖掘木材和烹饪食物, 然后进阶到更复杂的任务, 如对抗怪物和制作钻石工具。本文认为, 有效的终身学习智能体应具备与人类玩家相似的能力: (1) 根据其当前技能水平和世界状态提出合适的任务, 例如, 如果身处沙漠而非森林, 则先学习采集沙子和仙人掌, 再学习采集铁; (2) 根据环境反馈完善技能, 并将掌握的技能存入记忆, 以便未来在类似情境中复用 (例如, 与僵尸战斗类似于与蜘蛛战斗); (3) 以自我驱动的方式持续探索世界并寻求新任务。

为实现这些目标, 本文提出 VOYAGER, 这是首个由大语言模型驱动的具身终身学习智能体, 能够在《我的世界》中自主推动探索、掌握广泛技能, 并在无人干预的情况下持续做出新发现。VOYAGER 的实现依赖于三个关键模块 (图 2): 1) 一个最大化探索的自动课程; 2) 一个用于存储和检索复杂行为的技能库; 3) 一种新颖的迭代提示机制, 用于生成具身控制的可执行代码。本文选择代码而非低级电机指令作为动作空间, 因为程序能够自然表示时间上扩展且可组合的动作 [16, 22], 这对于《我的世界》中许多长时程任务至关重要。VOYAGER 通过提示和上下文学习 [36 – 38] 与黑盒大语言模型 (GPT-4[35]) 交互。本文方法无需访问模型参数, 也无需显式的基于梯度的训练或微调。

具体而言, VOYAGER 尝试解决由自动课程提出的逐步更难的任务, 该课程考虑了探索进度和智能体状态。课程由 GPT-4 基于“尽可能发现多样事物”的总体目标生成。这种方法可视为一种上下文形式的新颖性搜索 [39, 40]。VOYAGER 通过存储成功解决任务的动作程序, 逐步构建技能库。



图 3: 自动课程提出的任务。为简洁起见, 仅展示部分提示。完整提示结构见附录 A.3 节。

每个程序以其描述的嵌入为索引, 未来在相似情境中可被检索。复杂技能可通过组合更简单的程序来合成, 这使 VOYAGER 的能力随时间迅速复合增长, 并缓解了其他持续学习方法中的灾难性遗忘 [31, 32]。

然而, 大语言模型难以一次性稳定生成正确的动作代码 [41]。为应对这一挑战, 本文提出一种迭代提示机制: (1) 执行生成的程序, 从《我的世界》仿真环境中获取观测信息 (如物品栏列表和附近生物) 以及代码解释器的错误迹 (如有); (2) 将反馈纳入 GPT-4 的提示中, 进行下一轮代码优化; (3) 重复该过程, 直到自验证模块确认任务完成, 此时将程序提交至技能库 (如制作石铲和用剑战斗僵尸), 并向自动课程查询下一个里程碑 (图 2)。

实验表明, VOYAGER 展现出强大的上下文终身学习能力。它能够构建一个不断增长的技能库, 其中的动作程序可复用、可解释, 并能泛化到新任务。本文在开源《我的世界》人工智能框架 MineDojo[23] 中, 将 VOYAGER 与其他基于大语言模型的智能体技术 (如 ReAct[29]、Reflexion[30]、AutoGPT[28]) 进行了系统评估。VOYAGER 比先前的最先进方法多获得 $3.3\times$ 种独特物品, 解锁关键科技树里程碑的速度快 $15.3\times$ 倍, 且探索距离长 $2.3\times$ 倍。进一步证明, VOYAGER 能够在一个新的《我的世界》世界中利用已学技能库从零开始解决新任务, 而其他方法难以泛化。

2 方法

VOYAGER 由三个新颖组件构成: (1) 自动课程 (第 2.1 节), 为开放式探索提出目标; (2) 技能库 (第 2.2 节), 用于发展日益复杂的行为; (3) 迭代提示机制 (第 2.3 节), 生成用于具身控制的可执行代码。完整提示见附录 A 节。

2.1 自动课程

具身智能体在开放式环境中会遇到复杂度各异的目标。自动课程为开放式探索带来诸多益处, 既能确保学习过程富有挑战性但可控, 又能培养智能体以好奇心驱动的内生动机去学习和探索, 还能促进通用且灵活的问题解决策略的发展 [42–44]。我们的自动课程利用 GPT-4 所蕴含的互联网规模知识, 通过提示其持续提供新任务或挑战来运作。课程以自下而上的方式展开, 对探索进度和智能体当前状态具有高度的适应性和响应性 (图 3)。随着 VOYAGER 逐步迈向更难的自我驱动目标, 它自然而然地学会了多种技能, 例如“开采钻石”。

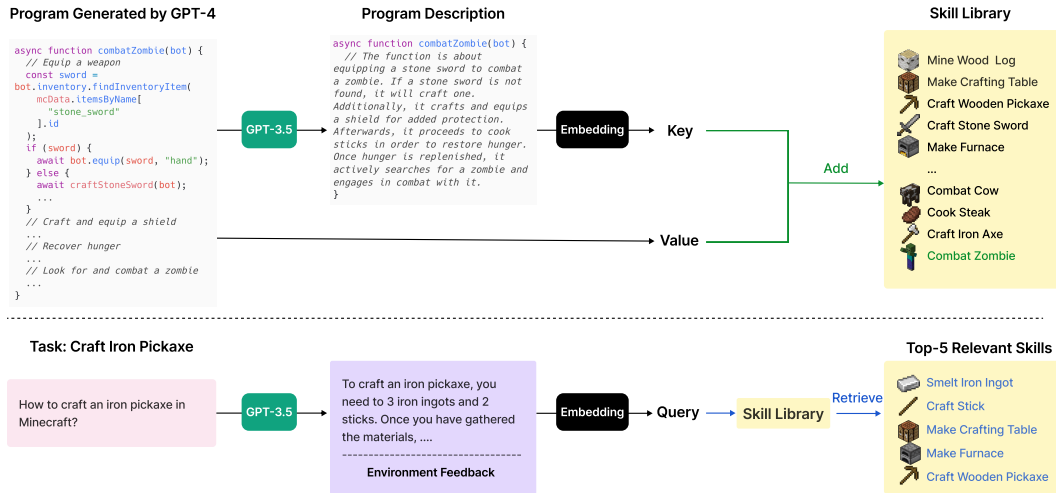


图 4：技能库。顶部：添加新技能。每当 GPT-4 生成并验证一项新技能时，我们将其添加到技能库中，该库以向量数据库表示。键是程序描述的嵌入向量（由 GPT-3.5 生成），值则是程序本身。底部：技能检索。当面对自动课程提出的新任务时，我们首先利用 GPT-3.5 生成解决该任务的通用建议，并将其与环境反馈结合作为查询上下文。随后，我们执行查询以识别最相关的 5 项技能。

输入给 GPT-4 的提示由以下几个部分组成：

- （1）鼓励多样化行为并施加约束的指令，例如“我的最终目标是尽可能发现多样的事物……下一个任务不应太难，因为我可能还没有必要的资源或尚未学会足够的技能来完成它。”；
- （2）智能体的当前状态，包括物品栏、装备、附近的方块和实体、生物群系、时间、生命值和饥饿值以及位置；
- （3）先前已完成和失败的任务，反映智能体当前的探索进度和能力边界；

- （4）**额外上下文**：我们还利用 GPT-3.5 根据智能体的当前状态和探索进度进行自我提问和自我回答。出于预算考虑，我们选择使用 GPT-3.5 而非 GPT-4 来处理标准自然语言处理任务。

2.2 技能库

随着自动课程持续提出日益复杂的任务，拥有一个技能库作为学习与演化的基础至关重要。受程序（Program）的通用性、可解释性和普适性启发 [45]，我们将每个技能表示为可执行代码，该代码为完成自动课程提出的特定任务搭建了时间扩展动作的脚手架。

输入到 GPT-4 的提示由以下部分组成：

- （1）代码生成指南，例如“你的函数将被复用于构建更复杂的函数。因此，你应使其通用且可复用。”；
- （2）控制原语应用程序编程接口（API），以及从技能库中检索的相关技能，这些对于上下文学习（In-Context Learning）[36 – 38] 的有效运作至关重要；
- （3）**上一轮生成的代码、环境反馈、执行错误和批评，基于这些 GPT-4 可以自我改进（第 2.3 节）**；
- （4）智能体当前状态，包括物品栏、装备、附近方块和实体、生物群系、时间、生命值和饥饿值条以及位置；

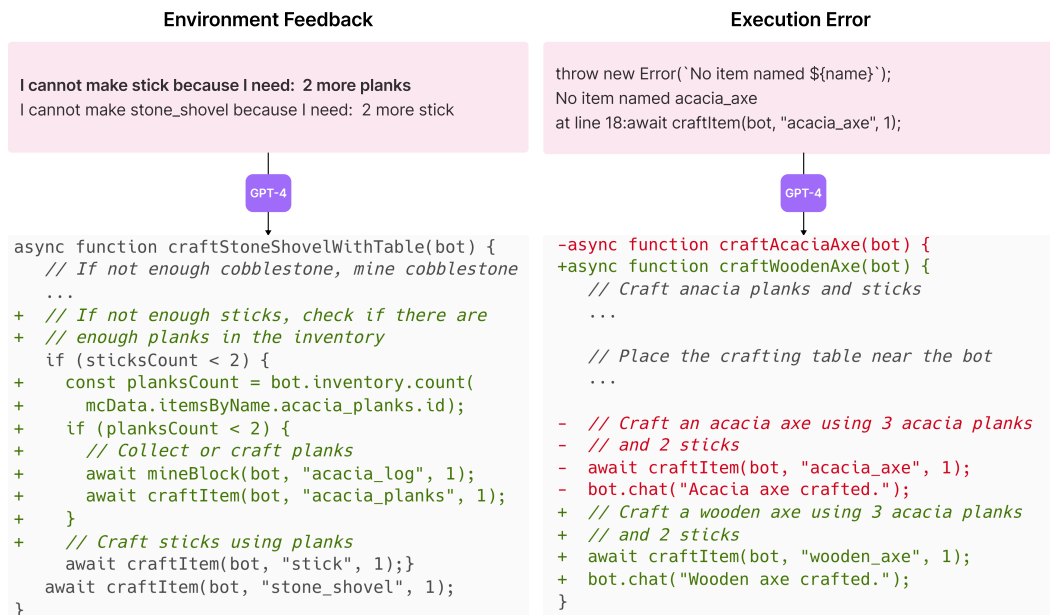


图 5：左：环境反馈。GPT-4 意识到在制作木棍之前还需要 2 块木板。右：执行错误。GPT-4 意识到应该制作木斧而不是金合欢斧，因为《我的世界》中不存在金合欢斧。为简洁起见，我们仅显示部分提示。代码生成的完整提示结构见附录第 A.4 节。

(5) 思维链提示 [46] 在代码生成前进行推理。

本文通过一种新颖的迭代提示机制（第 2.3 节）迭代优化程序，将其作为新技能纳入技能库，并以其描述的嵌入进行索引（图 4，上）。对于技能检索，本文利用自生成任务计划和环境反馈的嵌入查询技能库（图 4，下）。通过不断扩展和优化技能库，VOYAGER 能够在广泛的任务中学习、适应并表现出色，不断突破其在开放世界中的能力边界。

2.3 迭代提示机制

本文引入一种迭代提示机制，通过三种类型的反馈实现自我改进：

- (1) 环境反馈，展示程序执行的中间进展（图 5，左）。例如，“我无法制作铁胸甲，因为我需要：7 个以上的铁锭”指出了制作铁胸甲失败的原因。本文在控制原语应用程序编程接口中使用 `bot.chat()` 生成环境反馈，并提示 GPT-4 在代码生成过程中也使用该函数；
- (2) 来自程序解释器的执行错误，揭示程序中的无效操作或语法错误，对修复缺陷具有重要价值（图 5，右）；

(3) 用于检查任务成功与否的自我验证。本文并未为自动课程提出的每个新任务手动编写成功检查器，而是实例化另一个 GPT-4 智能体进行自我验证。通过向 GPT-4 提供 VOYAGER 的当前状态和任务，本文要求其充当批评者 [47 – 49]，告知程序是否完成任务。此外，如果任务失败，它会通过建议如何完成任务来提供批评意见（图 6）。因此，本文的自我验证比自我反思 [30] 更为全面，既检查成功，又反思错误。

在每轮代码生成过程中，我们执行生成的程序以从代码解释器获取环境反馈和执行错误，并将这些信息纳入 GPT-4 的提示中，用于下一轮代码优化。这一迭代过程持续进行，直到自我验证确认

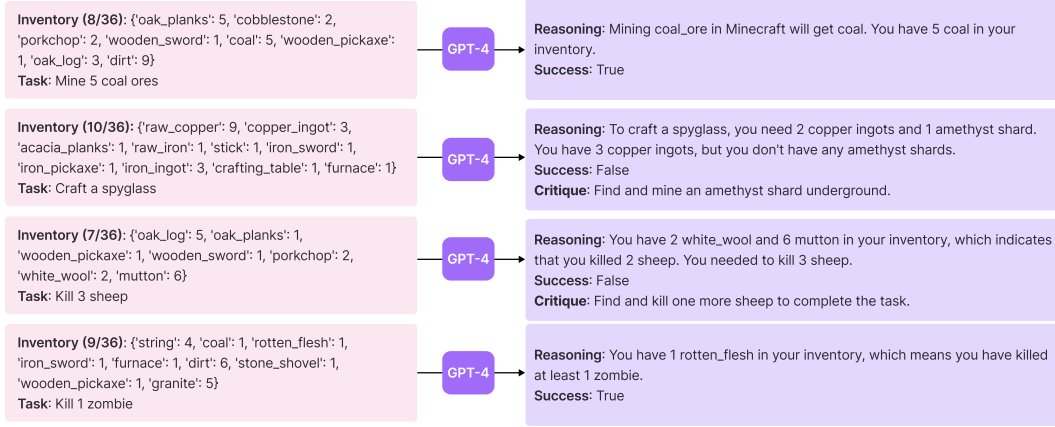


图 6: 自我验证示例。为简洁起见, 我们仅展示部分提示。完整提示结构见附录 A.5 节。

任务完成, 此时我们将这一新技能添加到技能库中, 并向自动课程请求一个新目标 (图 2)。如果智能体在 4 轮代码生成后仍陷入困境, 我们则向课程请求另一个任务。这种迭代提示方法显著提升了具身控制的程序合成能力, 使 VOYAGER 能够在无需人工干预的情况下持续获取多样技能。

3 实验

3.1 实验设置

我们利用 OpenAI 的 gpt-4-0314 [35] 和 gpt-3.5-turbo-0301 [50] 应用程序编程接口进行文本补全, 并使用 text-embedding-ada-002 [51] 应用程序编程接口进行文本嵌入。除自动课程外, 我们将所有温度参数设为 0, 自动课程使用温度 = 0.1 以鼓励任务多样性。我们的仿真环境基于 MineDojo [23] 构建, 并利用 Mineflayer [52] JavaScript 应用程序编程接口进行电机控制。更多细节见附录 B.1 节。

3.2 基线

由于目前没有可直接用于 Minecraft 的基于大语言模型的智能体, 我们尽力选择若干代表性算法作为基线。这些方法最初仅针对无具身的自然语言处理任务设计, 因此我们必须重新诠释它们, 使其能在 MineDojo 中执行并与我们的实验设置兼容: ReAct [29] 使用思维链提示 [46], 通过大语言模型生成推理迹和行动计划。我们为其提供环境反馈和智能体状态作为观测。

反思 [30] 构建于 ReAct [29] 之上, 并引入自我反思以推断更直观的未来行动。我们为其提供执行错误和我们的自我验证模块。

自动生成预训练变换器 [28] 是一款流行的软件工具, 通过将高层目标分解为多个子目标并以 ReAct 风格循环执行来自动化自然语言处理任务。我们使用生成式预训练变换器 4 进行任务分解, 并为其提供智能体状态、环境反馈和执行错误作为子目标执行的观测, 从而重新实现了自动生成预训练变换器。与 VOYAGER 相比, 自动生成预训练变换器缺乏用于积累知识的技能库、用于评估任务成功的自我验证, 以及用于开放式探索的自动课程。

请注意, 我们并未直接与那些以《我的世界》屏幕像素为输入并输出低级控制的先前方法进行比较 [53 – 55]。这并非同类比较, 因为我们依赖高级 Mineflayer [52] 应用程序编程接口来控制智能体。本研究的重点在于推动生成式预训练变换器 4 在终身具身智能体学习方面的极限, 而非解决三维感知或感觉运动控制问题。VOYAGER 是正交的, 可以与基于梯度的方法结合, 例如

表 1: 科技树掌握情况。分数表示三次总运行中成功的试验次数。0/3 表示该方法在最大提示迭代次数（160）内未能解锁科技树的某一等级。数字为三次试验平均的提示迭代次数。迭代次数越少，方法效率越高。

Method	Wooden Tool	Stone Tool	Iron Tool	Diamond Tool
ReAct [29]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
Reflexion [30]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28]	92 $\pm$ 72 (3/3)	94 $\pm$ 72 (3/3)	135 $\pm$ 103 (3/3)	N/A (0/3)
VOYAGER w/o Skill Library	7 $\pm$ 2 (3/3)	9 $\pm$ 4 (3/3)	29 $\pm$ 11 (3/3)	N/A (0/3)
VOYAGER (Ours)	6 $\pm$ 2 (3/3)	11 $\pm$ 2 (3/3)	21 $\pm$ 7 (3/3)	102 (1/3)

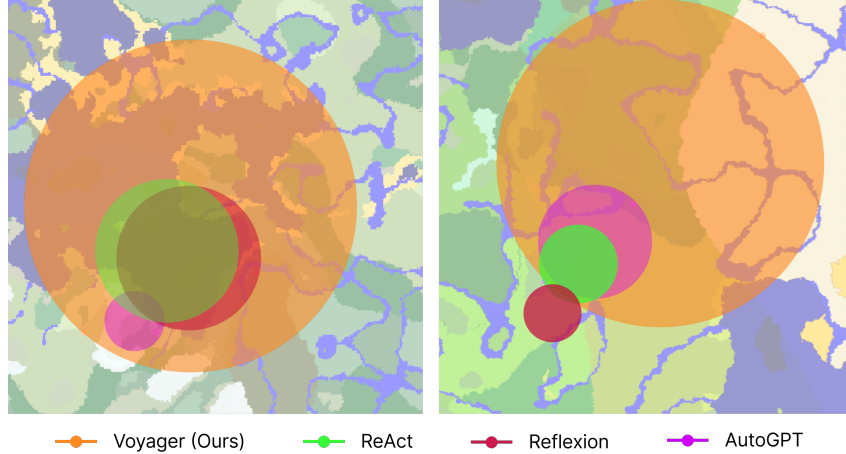


图 7: 地图覆盖范围: 《我的世界》地图的鸟瞰图。与基线相比, VOYAGER 能够穿越 2.3 $\times$ 更长的距离, 同时跨越多样地形。

VPT [8] 只要控制器提供代码应用程序编程接口。我们在表 A.2 中对 VOYAGER 与先前的《我的世界》智能体进行了系统级比较。

3.3 评估结果

本文系统评估了 VOYAGER 与基线方法在探索性能、科技树掌握程度、地图覆盖率以及在新世界中零样本泛化到新任务的能力。

显著更好的探索性能。探索性能结果如图 1 所示。VOYAGER 的优势在于其能够持续取得新进展, 在 160 次提示迭代内发现了 63 种独特物品, 3.3 $\times$ 与同类方法相比发现了许多新颖物品。另一方面, AutoGPT 在发现新物品方面明显滞后, 而 ReAct 和 Reflexion 由于开放式探索目标的抽象性, 在没有合适课程的情况下难以执行, 因此难以取得显著进展。

持续的科技树掌握。Minecraft 科技树测试了智能体制作和使用工具层级的能力。沿着科技树推进（木制工具 $\rightarrow$ 石制工具 $\rightarrow$ 铁制工具 $\rightarrow$ 钻石工具）需要智能体掌握系统性和组合性技能。与基线方法相比, VOYAGER 解锁木制层级 15.3 $\times$ 更快（以提示迭代次数计），解锁石制层级 8.5 $\times$ 更快, 解锁铁制层级 6.4 $\times$ 更快, 并且 VOYAGER 是唯一解锁科技树钻石层级的方法（图 2 和表 1）。这凸显了自动课程的有效性, 它持续呈现适当复杂度的挑战以促进智能体的进步。

广泛的地图遍历。VOYAGER 通过穿越各种地形, 能够比基线方法导航更长的距离 2.3 $\times$, 而基线智能体往往局限于局部区域, 这显著削弱了它们发现新知识的能力（图 7）。

表 2: 对未见任务的零样本泛化。分数表示三次总尝试中成功的试验次数。0/3 表示该方法在最大提示迭代次数（50 次）内未能解决任务。数字为三次试验的平均提示迭代次数。迭代次数越少，方法效率越高。

Method	Diamond Pickaxe	Golden Sword	Lava Bucket	Compass
ReAct [29]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
Reflexion [30]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28] w/ Our Skill Library	39 (1/3)	30 (1/3)	N/A (0/3)	30 (2/3)
VOYAGER w/o Skill Library	36 (2/3)	30 ± 9 (3/3)	27 ± 9 (3/3)	26 ± 3 (3/3)
VOYAGER (Ours)	19 ± 3 (3/3)	18 ± 7 (3/3)	21 ± 5 (3/3)	18 ± 2 (3/3)

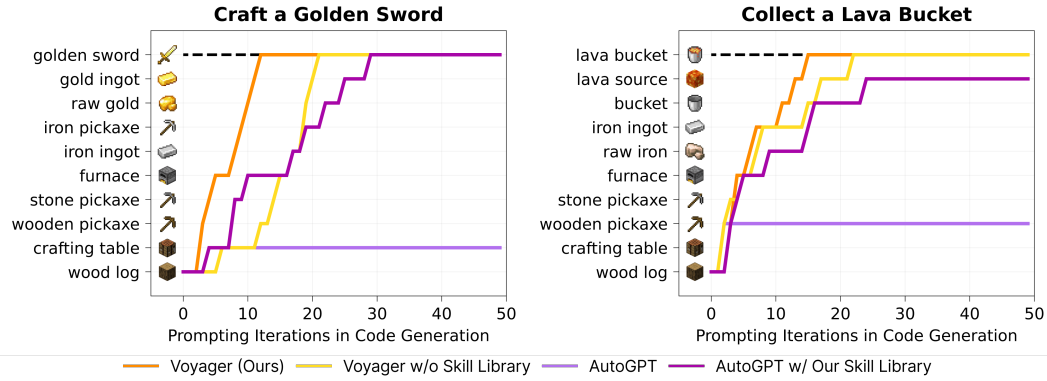


图 8: 对未见任务的零样本泛化。我们可视化了每种方法在两个任务上的中间进展。其他两个任务见附录第 B.4.3 节。我们没有绘制 ReAct 和 Reflexion，因为它们没有取得任何有意义的进展。

对未见任务的高效零样本泛化。为了评估零样本泛化，我们清空智能体的物品栏，将其重置到一个新实例化的世界，并用未见任务进行测试。对于 VOYAGER 和 AutoGPT，我们利用 GPT-4 将任务分解为一系列子目标。表 2 和图 8 显示，VOYAGER 能够持续解决所有任务，而基线方法在 50 次提示迭代内无法解决任何任务。值得注意的是，我们通过终身学习构建的技能库不仅提升了 VOYAGER 的性能，还提升了 AutoGPT 的性能。这表明技能库是一种通用工具，可以方便地被其他方法使用，有效地作为即插即用的资产来提升性能。

3.4 消融研究

我们对 VOYAGER 中的 6 个设计选择（自动课程、技能库、环境反馈、执行错误、自我验证和用于代码生成的 GPT-4）进行了消融，并研究了它们对探索性能的影响（每个消融变体的详细信息见附录第 B.3 节）。结果如图 9 所示。我们重点强调以下发现：

- **自动课程对于智能体的持续进步至关重要。**如果将课程替换为随机课程，发现的物品数量会下降 93%，因为某些任务如果顺序不当可能会过于困难。另一方面，手动设计的课程需要大量的 Minecraft 专业知识，并且没有考虑智能体的实时情况。在实验结果中，它不如我们的自动课程。
- **VOYAGER 无技能库时，后期表现出趋于停滞的倾向。**这凸显了技能库在 VOYAGER 中的关键作用。它有助于创建更复杂的动作，并通过鼓励在旧技能基础上构建新技能，稳步拓展智能体的能力边界。

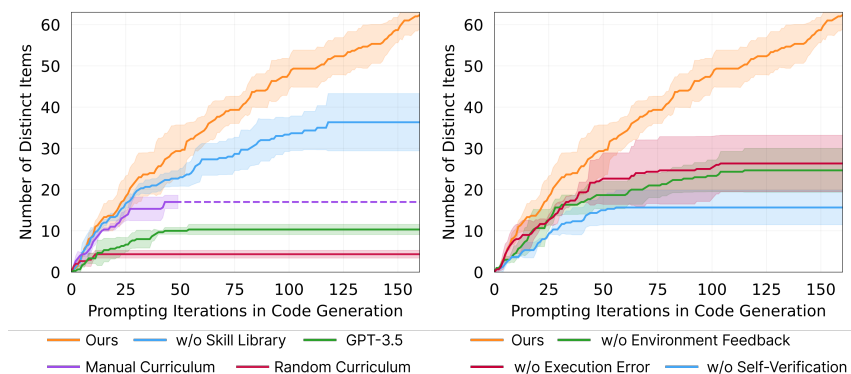


图 9：左：自动课程、技能库和 GPT-4 的消融研究。GPT-3.5 表示用 GPT-3.5 替换 GPT-4 进行代码生成。VOYAGER 优于所有替代方案，证明了每个组件的关键作用。右：迭代提示机制的消融研究。VOYAGER 超越了所有其他选项，从而凸显了每种反馈在迭代提示机制中的重要性。



图 10：VOYAGER 在人类反馈下构建 3D 结构。从左到右展示了整合人类输入的建筑进度。

- 自我验证在所有反馈类型中最为重要。移除该模块会导致发现物品数量显著下降（-73%）。自我验证是决定何时转向新任务或重新尝试先前失败任务的关键机制。
- GPT-4 在代码生成方面显著优于 GPT-3.5，并多获得 $5.7\times$ 个独特物品，因为 GPT-4 在编码能力上实现了质的飞跃。这一发现与文献 [56, 57] 中的近期研究相印证。

3.5 来自人类的多模态反馈

VOYAGER 目前不支持视觉感知，因为撰写本文时可用的 GPT-4 API 版本仅支持文本。然而，VOYAGER 有潜力通过多模态感知模型 [58, 59] 进行增强，以实现更出色的任务。我们证明，在人类反馈下，VOYAGER 能够在《我的世界》中构建复杂的 3D 结构，例如下界传送门和房屋（图 10）。整合人类反馈有两种方式：

- (1) 人类作为批评者（相当于 VOYAGER 的自我验证模块）：人类向 VOYAGER 提供视觉批评，使其能够修改上一轮的代码。这种反馈对于纠正 VOYAGER 无法直接感知的 3D 结构空间细节中的某些错误至关重要。
- (2) 人类作为课程（相当于“航海家”的自动课程模块）：人类将复杂的建造任务分解为更小的步骤，引导“航海家”逐步完成。这种方法提升了“航海家”处理更复杂三维建造任务的能力。

4 局限性与未来工作

成本。GPT-4 应用程序编程接口 (API) 产生显著成本。它比 GPT-3.5 贵 $15\times$ 。然而,“航海家”需要 GPT-4 带来的代码生成质量的飞跃 (图 9), 这是 GPT-3.5 和开源大语言模型无法提供的 [60]。

不准确性。尽管采用了迭代提示机制, 仍存在智能体陷入困境、无法生成正确技能的情况。自动课程具有灵活性, 可以在稍后时间重新尝试该任务。偶尔, 自我验证模块也可能失败, 例如未将蜘蛛丝识别为击败蜘蛛的成功信号。

幻觉。自动课程偶尔会提出无法实现的任务。例如, 它可能要求智能体制作“铜剑”或“铜胸甲”, 而这些物品在游戏中并不存在。幻觉也发生在代码生成过程中。例如, GPT-4 倾向于使用圆石作为燃料输入, 尽管圆石在游戏中是无效的燃料来源。此外, 它可能调用所提供的控制原语应用程序编程接口 (API) 中不存在的函数, 导致代码执行错误。

我们相信, GPT API 模型的改进以及针对开源大语言模型微调的新技术将在未来克服这些局限性。

5 相关工作

《我的世界》中的决策智能体。《我的世界》是一个开放式的三维世界, 其游戏机制极为灵活, 支持广泛的活动。基于著名的《我的世界》基准 [23, 61 – 65], 《我的世界》学习算法可分为两类: 1) 低层控制器: 许多先前的工作利用分层强化学习从人类示范中学习 [66 – 68]。Kanitscheider 等人 [14] 设计了一种基于成功率的课程, 但其目标仅限于精选物品。MineDojo [23] 和 VPT [8] 利用 YouTube 视频进行大规模预训练。另一方面, DreamerV3 [69] 学习世界模型以探索环境并收集钻石。2) 高层规划器: Volum 等人 [70] 利用 Codex [41] 的少样本提示生成可执行策略, 但需要额外的人类交互。近期工作利用大语言模型作为《我的世界》中的高层规划器, 通过按照《我的世界》配方将高层任务分解为若干子目标 [55, 53, 71], 因此缺乏充分的探索灵活性。与后者类似, VOYAGER 也通过提示 GPT-4 使用大语言模型作为高层规划器, 并按照 Volum 等人 [70] 的方法利用 Mineflayer [52] 作为低层控制器。与先前工作不同, VOYAGER 采用了一种由好奇心驱动的自下而上自动课程, 因此实现了开放式探索。

用于智能体规划的大语言模型。受大语言模型 (LLM) 强大的涌现能力 (如零样本提示和复杂推理 [72, 37, 38, 36, 73, 74]) 的启发, 具身智能体研究 [75 – 78] 在利用大语言模型进行规划方面显著增加。近期工作大致可分为两类。1) 用于机器人学习的大语言模型: 许多先前工作应用大语言模型为机器人规划生成子目标 [27, 27, 25, 79, 80]。内心独白 (Inner Monologue) [26] 将环境反馈纳入基于大语言模型的机器人规划。代码即策略 (Code as Policies) [16] 和 ProgPrompt [22] 直接利用大语言模型生成可执行的机器人策略。VIMA [19] 和 PaLM-E [59] 微调预训练大语言模型以支持多模态提示。2) 用于文本智能体的大语言模型: ReAct [29] 利用思维链提示 [46], 通过大语言模型生成推理迹和任务特定动作。Reflexion [30] 在 ReAct [29] 基础上引入自我反思以增强推理。AutoGPT [28] 是一种流行工具, 通过为完成高层目标构建多个子目标的课程, 并结合 ReAct [29] 的推理与行动循环来自动化自然语言处理任务。DERA [81] 将任务构建为两个 GPT-4 [35] 智能体之间的对话。生成式智能体 (Generative Agents) [82] 利用 ChatGPT [50] 通过将智能体经验存储为记忆并检索用于规划来模拟人类行为, 但其智能体动作不可执行。SPRING [83] 是一项同期工作, 使用 GPT-4 从游戏手册中提取游戏机制, 据此回答有向无环图中排列的问题并预测下一步动作。所有这些工作都缺乏用于开发更复杂行为的技能库, 而技能库是 VOYAGER 在终身学习中取得成功的关键组成部分。

代码生成与执行。代码生成一直是自然语言处理 (NLP) [41, 84, 85, 73, 37] 中长期存在的挑战, 许多工作利用执行结果来改进程序

综合。执行引导的方法利用中间执行结果来指导程序搜索 [86 – 88]。另一条研究路线利用多数投票根据候选程序的执行性能进行选择 [89, 90]。此外，LEVER [91] 训练一个验证器，根据执行结果区分并拒绝不正确的程序。另一方面，CLAIRIFY [92] 为规划化学实验生成代码，并利用基于规则的验证器迭代地向大语言模型提供错误反馈。VOYAGER 与这些工作的区别在于，它将环境反馈、执行错误和自我验证（用于评估任务成功）整合到一个迭代提示机制中，用于具身控制。

6 结论

在这项工作中，我们介绍了 VOYAGER，这是第一个由大语言模型驱动的具身终身学习智能体，它利用 GPT-4 持续探索世界，不断开发日益复杂的技能，并在没有人类干预的情况下持续做出新发现。VOYAGER 在发现新物品、解锁 Minecraft 科技树、穿越多样地形以及将其学到的技能库应用于新实例化世界中的未见任务方面表现出卓越的性能。VOYAGER 作为开发强大通用智能体的起点，无需调整模型参数。

7 更广泛影响

我们的研究是在 Minecraft 中进行的，这是一个安全无害的 3D 视频游戏环境。虽然 VOYAGER 被设计为可普遍适用于其他领域，例如机器人技术，但其在物理机器人上的应用需要额外的关注以及人类实施安全约束，以确保负责任和安全的部署。

8 致谢

我们非常感谢 Ziming Zhu、Kaiyu Yang、Rafał Kocielnik、Colin White、Or Sharir、Sahin Lale、De-An Huang、Jean Kossaifi、Yuncong Yang、Charles Zhang、Minchao Huang 以及许多其他同事和朋友的有益反馈和深刻讨论。这项工作是在 Guanzhi Wang 于 NVIDIA 实习期间完成的。Guanzhi Wang 得到了加州理工学院计算与数学科学 Kortschak 奖学金的支持。

参考文献

- [1] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Daniel Gordon, Yuke Zhu, Abhinav Gupta, and Ali Farhadi. Ai2-thor: An interactive 3d environment for visual ai. *arXiv preprint arXiv: Arxiv-1712.05474*, 2017.
- [2] Manolis Savva, Jitendra Malik, Devi Parikh, Dhruv Batra, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, and Vladlen Koltun. Habitat: A platform for embodied AI research. In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*, pages 9338–9346. IEEE, 2019.
- [3] Yuke Zhu, Josiah Wong, Ajay Mandlekar, and Roberto Martín-Martín. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv: Arxiv-2009.12293*, 2020.
- [4] Fei Xia, William B. Shen, Chengshu Li, Priya Kasimbeg, Micael Tchampi, Alexander Toshev, Li Fei-Fei, Roberto Martín-Martín, and Silvio Savarese. Interactive gibbon benchmark (igibson 0.5): A benchmark for interactive navigation in cluttered environments. *arXiv preprint arXiv: Arxiv-1910.14442*, 2019.
- [5] Bokui Shen, Fei Xia, Chengshu Li, Roberto Martín-Martín, Linxi Fan, Guanzhi Wang, Claudia Pérez-D’Arpino, Shyamal Buch, Sanjana Srivastava, Lyne P. Tchampi, Micael E. Tchampi, Kent Vainio, Josiah Wong, Li Fei-Fei, and Silvio Savarese. igibson 1.0: a simulation environment for interactive tasks in large realistic scenes. *arXiv preprint arXiv: Arxiv-2012.02924*, 2020.

- [6] Jens Kober, J Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013.
- [7] Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage, and Anil Anthony Bharath. Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine*, 34(6):26–38, 2017.
- [8] Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv: Arxiv-2206.11795*, 2022.
- [9] DeepMind Interactive Agents Team, Josh Abramson, Arun Ahuja, Arthur Brussee, Federico Carnevale, Mary Cassin, Felix Fischer, Petko Georgiev, Alex Goldin, Mansi Gupta, Tim Harley, Felix Hill, Peter C Humphreys, Alden Hung, Jessica Landon, Timothy Lillicrap, Hamza Merzic, Alistair Muldal, Adam Santoro, Guy Scully, Tamara von Glehn, Greg Wayne, Nathaniel Wong, Chen Yan, and Rui Zhu. Creating multimodal interactive agents with imitation and self-supervised learning. *arXiv preprint arXiv: Arxiv-2112.03763*, 2021.
- [10] Oriol Vinyals, Igor Babuschkin, Junyoung Chung, Michael Mathieu, Max Jaderberg, Wojciech M Czarnecki, Andrew Dudzik, Aja Huang, Petko Georgiev, Richard Powell, et al. Alphastar: Mastering the real-time strategy game starcraft ii. *DeepMind blog*, 2, 2019.
- [11] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. Go-explore: a new approach for hard-exploration problems. *arXiv preprint arXiv: Arxiv-1901.10995*, 2019.
- [12] Joost Huizinga and Jeff Clune. Evolving multimodal robot behavior via many stepping stones with the combinatorial multiobjective evolutionary algorithm. *Evolutionary computation*, 30(2):131–164, 2022.
- [13] Rui Wang, Joel Lehman, Aditya Rawal, Jiale Zhi, Yulun Li, Jeffrey Clune, and Kenneth O. Stanley. Enhanced POET: open-ended reinforcement learning through unbounded invention of learning challenges and their solutions. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 9940–9951. PMLR, 2020.
- [14] Ingmar Kanitscheider, Joost Huizinga, David Farhi, William Hebgren Guss, Brandon Houghton, Raul Sampedro, Peter Zhokhov, Bowen Baker, Adrien Ecoffet, Jie Tang, Oleg Klimov, and Jeff Clune. Multi-task curriculum learning in a complex, visual, hard-exploration domain: Minecraft. *arXiv preprint arXiv: Arxiv-2106.14876*, 2021.
- [15] Michael Dennis, Natasha Jaques, Eugene Vinitzky, Alexandre M. Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- [16] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv: Arxiv-2209.07753*, 2022.
- [17] Shao-Hua Sun, Te-Lin Wu, and Joseph J. Lim. Program guided agent. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.
- [18] Zelin Zhao, Karan Samel, Binghong Chen, and Le Song. Proto: Program-guided transformer for program-guided tasks. In Marc’Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 17021–17036, 2021.
- [19] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi (Jim) Fan. Vima: General robot manipulation with multimodal prompts. *ARXIV.ORG*, 2022.

- [20] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Cliport: What and where pathways for robotic manipulation. *arXiv preprint arXiv: Arxiv-2109.12098*, 2021.
- [21] Linxi Fan, Guanzhi Wang, De-An Huang, Zhiding Yu, Li Fei-Fei, Yuke Zhu, and Animashree Anandkumar. SECANT: self-expert cloning for zero-shot generalization of visual policies. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 3088–3099. PMLR, 2021.
- [22] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv: Arxiv-2209.11302*, 2022.
- [23] Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. *arXiv preprint arXiv: Arxiv-2206.08853*, 2022.
- [24] Andy Zeng, Adrian Wong, Stefan Welker, Krzysztof Choromanski, Federico Tombari, Aveek Purohit, Michael Ryoo, Vikas Sindhwani, Johnny Lee, Vincent Vanhoucke, and Pete Florence. Socratic models: Composing zero-shot multimodal reasoning with language. *arXiv preprint arXiv: Arxiv-2204.00598*, 2022.
- [25] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, and Mengyuan Yan. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv: Arxiv-2204.01691*, 2022.
- [26] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Noah Brown, Tomas Jackson, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv: Arxiv-2207.05608*, 2022.
- [27] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato, editors, *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pages 9118–9147. PMLR, 2022.
- [28] Significant-gravitas/auto-gpt: An experimental open-source attempt to make gpt-4 fully autonomous., 2023.
- [29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv: Arxiv-2210.03629*, 2022.
- [30] Noah Shinn, Beck Labash, and Ashwin Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *arXiv preprint arXiv: Arxiv-2303.11366*, 2023.
- [31] German Ignacio Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.
- [32] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning: Theory, method and application. *arXiv preprint arXiv: Arxiv-2302.00487*, 2023.
- [33] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv: Arxiv-1312.5602*, 2013.

- [34] OpenAI, :, Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębniak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique P. d. O. Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, and Susan Zhang. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv: Arxiv-1912.06680*, 2019.
- [35] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv: Arxiv-2303.08774*, 2023.
- [36] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *arXiv preprint arXiv: Arxiv-2206.07682*, 2022.
- [37] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- [38] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67, 2020.
- [39] Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.
- [40] Edoardo Conti, Vashisht Madhavan, Felipe Petroski Such, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. Improving exploration in evolution strategies for deep reinforcement learning via a population of novelty-seeking agents. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 5032–5043, 2018.
- [41] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv: Arxiv-2107.03374*, 2021.
- [42] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. Paired open-ended trailblazer (poet): Endlessly generating increasingly complex and diverse learning environments and their solutions. *arXiv preprint arXiv: Arxiv-1901.01753*, 2019.
- [43] Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. Automatic curriculum learning for deep RL: A short survey. In Christian Bessiere, editor, *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*, pages 4819–4825. ijcai.org, 2020.

- [44] Sébastien Forestier, Rémy Portelas, Yoan Mollard, and Pierre-Yves Oudeyer. Intrinsically motivated goal exploration processes with automatic curriculum learning. *The Journal of Machine Learning Research*, 23(1):6818–6858, 2022.
- [45] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sable-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. Dreamcoder: Growing generalizable, interpretable knowledge with wake-sleep bayesian program learning. *arXiv preprint arXiv: 2006.08381*, 2020.
- [46] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv: 2201.11903*, 2022.
- [47] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In Maria-Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1928–1937. JMLR.org, 2016.
- [48] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv: 1707.06347*, 2017.
- [49] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In Yoshua Bengio and Yann LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.
- [50] Introducing chatgpt, 2022.
- [51] New and improved embedding model, 2022.
- [52] PrismarineJS. Prismarinejs/mineflayer: Create minecraft bots with a powerful, stable, and high level javascript api., 2013.
- [53] Kolby Nottingham, Prithviraj Ammanabrolu, Alane Suhr, Yejin Choi, Hanna Hajishirzi, Sameer Singh, and Roy Fox. Do embodied agents dream of pixelated sheep?: Embodied decision making using language guided world modelling. *ARXIV.ORG*, 2023.
- [54] Shaofei Cai, Zihao Wang, Xiaojian Ma, Anji Liu, and Yitao Liang. Open-world multi-task control through goal-aware representation learning and adaptive horizon prediction. *arXiv preprint arXiv: 2301.10034*, 2023.
- [55] Zihao Wang, Shaofei Cai, Anji Liu, Xiaojian Ma, and Yitao Liang. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents. *arXiv preprint arXiv: 2302.01560*, 2023.
- [56] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv: 2303.12712*, 2023.
- [57] Yiheng Liu, Tianle Han, Siyuan Ma, Jiayue Zhang, Yuanyuan Yang, Jiaming Tian, Hao He, Antong Li, Mengshen He, Zhengliang Liu, Zihao Wu, Dajiang Zhu, Xiang Li, Ning Qiang, Dingang Shen, Tianming Liu, and Bao Ge. Summary of chatgpt/gpt-4 research and perspective towards the future of large language models. *arXiv preprint arXiv: 2304.01852*, 2023.
- [58] Shikun Liu, Linxi Fan, Edward Johns, Zhiding Yu, Chaowei Xiao, and Anima Anandkumar. Prism: A vision-language model with an ensemble of experts. *arXiv preprint arXiv: 2303.02506*, 2023.

- [59] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv: Arxiv-2303.03378*, 2023.
- [60] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *arXiv preprint arXiv: Arxiv-2302.13971*, 2023.
- [61] William H. Guss, Brandon Houghton, Nicholay Topin, Phillip Wang, Cayden Codel, Manuela Veloso, and Ruslan Salakhutdinov. Minerl: A large-scale dataset of minecraft demonstrations. In Sarit Kraus, editor, *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, pages 2442–2448. ijcai.org, 2019.
- [62] William H. Guss, Cayden Codel, Katja Hofmann, Brandon Houghton, Noboru Kuno, Stephanie Milani, Sharada Mohanty, Diego Perez Liebana, Ruslan Salakhutdinov, Nicholay Topin, Manuela Veloso, and Phillip Wang. The minerl 2019 competition on sample efficient reinforcement learning using human priors. *arXiv preprint arXiv: Arxiv-1904.10079*, 2019.
- [63] William H. Guss, Mario Yncente Castro, Sam Devlin, Brandon Houghton, Noboru Sean Kuno, Crissman Loomis, Stephanie Milani, Sharada Mohanty, Keisuke Nakata, Ruslan Salakhutdinov, John Schulman, Shinya Shiroshita, Nicholay Topin, Avinash Ummadisingu, and Oriol Vinyals. The minerl 2020 competition on sample efficient reinforcement learning using human priors. *arXiv preprint arXiv: Arxiv-2101.11071*, 2021.
- [64] Anssi Kanervisto, Stephanie Milani, Karolis Ramanauskas, Nicholay Topin, Zichuan Lin, Junyou Li, Jianing Shi, Deheng Ye, Qiang Fu, Wei Yang, Weijun Hong, Zhongyue Huang, Haicheng Chen, Guangjun Zeng, Yue Lin, Vincent Micheli, Eloi Alonso, François Fleuret, Alexander Nikulin, Yury Belousov, Oleg Svidchenko, and Aleksei Shpilman. Minerl diamond 2021 competition: Overview, results, and lessons learned. *arXiv preprint arXiv: Arxiv-2202.10583*, 2022.
- [65] Matthew Johnson, Katja Hofmann, Tim Hutton, and David Bignell. The malmo platform for artificial intelligence experimentation. In Subbarao Kambhampati, editor, *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI 2016, New York, NY, USA, 9-15 July 2016*, pages 4246–4247. IJCAI/AAAI Press, 2016.
- [66] Zichuan Lin, Junyou Li, Jianing Shi, Deheng Ye, Qiang Fu, and Wei Yang. Juwu-mc: Playing minecraft with sample-efficient hierarchical reinforcement learning. *arXiv preprint arXiv: Arxiv-2112.04907*, 2021.
- [67] Hangyu Mao, Chao Wang, Xiaotian Hao, Yihuan Mao, Yiming Lu, Chengjie Wu, Jianye Hao, Dong Li, and Pingzhong Tang. Seihai: A sample-efficient hierarchical ai for the minerl competition. *arXiv preprint arXiv: Arxiv-2111.08857*, 2021.
- [68] Alexey Skrynnik, Aleksey Staroverov, Ermek Aitygulov, Kirill Aksenov, Vasili Davydov, and Aleksandr I. Panov. Hierarchical deep q-network from imperfect demonstrations in minecraft. *Cogn. Syst. Res.*, 65:74–78, 2021.
- [69] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv: Arxiv-2301.04104*, 2023.
- [70] Ryan Volum, Sudha Rao, Michael Xu, Gabriel DesGarennes, Chris Brockett, Benjamin Van Durme, Olivia Deng, Akanksha Malhotra, and Bill Dolan. Craft an iron sword: Dynamically generating interactive game characters by prompting large language models tuned on code. In *Proceedings of the 3rd Wordplay: When Language Meets Games Workshop (Wordplay 2022)*, pages 25–43, Seattle, United States, 2022. Association for Computational Linguistics.
- [71] Haoqi Yuan, Chi Zhang, Hongcheng Wang, Feiyang Xie, Penglin Cai, Hao Dong, and Zongqing Lu. Plan4mc: Skill reinforcement learning and planning for open-world minecraft tasks. *arXiv preprint arXiv: 2303.16563*, 2023.

- [72] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models. *arXiv preprint arXiv: Arxiv-2108.07258*, 2021.
- [73] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv: Arxiv-2204.02311*, 2022.
- [74] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. Scaling instruction-finetuned language models. *arXiv preprint arXiv: Arxiv-2210.11416*, 2022.
- [75] Jiafei Duan, Samson Yu, Hui Li Tan, Hongyuan Zhu, and Cheston Tan. A survey of embodied AI: from simulators to research tasks. *IEEE Trans. Emerg. Top. Comput. Intell.*, 6(2):230–244, 2022.
- [76] Dhruv Batra, Angel X. Chang, Sonia Chernova, Andrew J. Davison, Jia Deng, Vladlen Koltun, Sergey Levine, Jitendra Malik, Igor Mordatch, Roozbeh Mottaghi, Manolis Savva, and Hao Su. Rearrangement: A challenge for embodied ai. *arXiv preprint arXiv: Arxiv-2011.01975*, 2020.
- [77] Harish Ravichandar, Athanasios S Polydoros, Sonia Chernova, and Aude Billard. Recent advances in robot learning from demonstration. *Annual review of control, robotics, and autonomous systems*, 3:297–330, 2020.
- [78] Jack Collins, Shelvin Chand, Anthony Vanderkop, and David Howard. A review of physics simulators for robotic applications. *IEEE Access*, 9:51416–51431, 2021.
- [79] So Yeon Min, Devendra Singh Chaplot, Pradeep Ravikumar, Yonatan Bisk, and R. Salakhutdinov. Film: Following instructions in language with modular methods. *International Conference on Learning Representations*, 2021.

- [80] Valts Blukis, Chris Paxton, Dieter Fox, Animesh Garg, and Yoav Artzi. A persistent spatial semantic representation for high-level natural language instruction execution. In *5th Annual Conference on Robot Learning*, 2021.
- [81] Varun Nair, Elliot Schumacher, Geoffrey Tso, and Anitha Kannan. Dera: Enhancing large language model completions with dialog-enabled resolving agents. *arXiv preprint arXiv: Arxiv-2303.17071*, 2023.
- [82] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv: Arxiv-2304.03442*, 2023.
- [83] Yue Wu, Shrimai Prabhumoye, So Yeon Min, Yonatan Bisk, Ruslan Salakhutdinov, Amos Azaria, Tom Mitchell, and Yuanzhi Li. Spring: Gpt-4 out-performs rl algorithms by studying papers and reasoning. *arXiv preprint arXiv: 2305.15486*, 2023.
- [84] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. A conversational paradigm for program synthesis. *arXiv preprint arXiv: Arxiv-2203.13474*, 2022.
- [85] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *arXiv preprint arXiv: Arxiv-2207.01780*, 2022.
- [86] Xinyun Chen, Chang Liu, and Dawn Song. Execution-guided neural program synthesis. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.
- [87] Xinyun Chen, Dawn Song, and Yuandong Tian. Latent execution for neural program synthesis. *arXiv preprint arXiv: Arxiv-2107.00101*, 2021.
- [88] Kevin Ellis, Maxwell I. Nye, Yewen Pu, Felix Sosa, Josh Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a REPL. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d’Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 9165–9174, 2019.
- [89] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *arXiv preprint arXiv: Arxiv-2203.07814*, 2022.
- [90] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv: Arxiv-2110.14168*, 2021.
- [91] Ansong Ni, Srinu Iyer, Dragomir Radev, Ves Stoyanov, Wen tau Yih, Sida I. Wang, and Xi Victoria Lin. Lever: Learning to verify language-to-code generation with execution. *arXiv preprint arXiv: Arxiv-2302.08468*, 2023.
- [92] Marta Skreta, Naruki Yoshikawa, Sebastian Arellano-Rubach, Zhi Ji, Lasse Bjørn Kristensen, Kourosh Darvish, Alán Aspuru-Guzik, Florian Shkurti, and Animesh Garg. Errors are useful prompts: Instruction guided task programming with verifier-assisted iterative prompting. *arXiv preprint arXiv: Arxiv-2303.14100*, 2023.

A 方法

A.1 VOYAGER 算法

Pseudocode 1: VOYAGER algorithm.

定义 voyager(环境, # 使用代码作为动作空间的环境 curriculum_agent, # 用于提出下一任务的课程代理 action_agent, # 用于代码生成的动作代理 critic_agent, # 用于自我验证的评论代理 skill_manager, # 用于添加新技能和技能检索的技能管理器):

```
agent_state = environment.reset()
while True:
    exploration_progress = (
        curriculum_agent.get_exploration_progress(
            curriculum_agent.get_completed_tasks(),
            curriculum_agent.get_failed_tasks(),
        )
    )
    task = curriculum_agent.propose_next_task(
        agent_state, exploration_progress) 代码 = 无 environment_feedback = 无 execution_errors
        = 无 评论 = 无 成功 = 假 # 在进入下一任务前最多尝试 4 轮, 循环 i 在范围 (4) 内:

    skills = skill_manager.retrieve_skills(
        task, environment_feedback
    )
    code = action_agent.generate_code(
        task,
        code,
        environment_feedback,
        execution_errors,
        critique,
        skills,
    )
    (
        agent_state,
        environment_feedback,
        execution_errors,
    ) = environment.step(code)
    success, critique = critic_agent.check_task_success(
        task, agent_state
    )
    if success:
        如果成功则中断:
            skill_manager.add_skill(code)
            curriculum_agent.add_completed_task(task)
    else:
        curriculum_agent.add_failed_task(task)
```

A.2 提示

GPT-4 和 GPT-3.5 允许用户在三个选项之间指定每条提示消息的角色:

- 系统：一条高层指令，在整个对话过程中引导模型行为。它为交互设定整体基调和目标。
- 用户：指导助手进行下一步即时响应的详细指令。
- 助手：模型生成的响应消息。

详见 <https://platform.openai.com/docs/guides/chat/introduction>。

为节省令牌用量，我们不进行多轮对话，而是将系统提示与用户提示拼接，以获取每个助手的响应。

A.3 自动课程

A.3.1 提示中的组件

输入给 GPT-4 的提示由以下几个部分组成：

- (1) 鼓励多样化行为并施加约束（使所提任务可实现且可验证）的指令：完整提示见第 A.3.4 节；
 - (2) 智能体的当前状态：
 - **物品栏**：一个包含物品及其数量的字典，例如，{'圆石': 4, '熔炉': 1, 'stone_pickaxe': 1, 'oak_planks': 7, '泥土': 6, 'wooden_pickaxe': 1, 'crafting_table': 1, 'raw_iron': 4, '煤炭': 1}；
 - **装备**：智能体装备的盔甲或武器；
 - **附近方块**：智能体 32 格距离内的方块名称集合，例如，'泥土'，'水'，'spruce_planks'，'grass_block'，'dirt_path'，'sugar_cane'，'蕨'；
 - **最近看到的其他方块**：不在附近也不在物品栏中的方块；
 - **附近实体**：距离智能体 32 个方块范围内的实体名称集合，例如“猪”、“猫”、“村民”、“僵尸”；
 - **智能体可见的箱子列表**：箱子是智能体可以存放物品的外部容器。若箱子此前未被打开，其内容显示为“未知”。否则，箱子内每件物品都会显示给智能体。
 - **生物群系**：例如“平原”、'flower_forest'，“草甸”、“河流”、“海滩”、“森林”、'snowy_slopes'，'frozen_peaks'，'old_growth_birch_forest'，“海洋”、'sunflower_plains'，'stony_shore'；
 - **时间**：以下之一：“日出”、“白天”、“正午”、“日落”、“夜晚”、“午夜”；
 - **生命值与饥饿值**：最大值为 20；
 - **位置**：智能体在《我的世界》世界中位置的三维坐标 (x, y, z) ；
- (3) 先前已完成和失败的任务；
 - (4) 额外上下文：见第 A.3.2 节；
 - (5) 思维链提示 [46] 作为响应：我们要求 GPT-4 首先推理当前进度，然后建议下一个任务。

A.3.2 额外上下文

我们利用 GPT-3.5 进行自我提问以提供额外上下文。每个问题与一个概念配对，该概念用于从维基知识库 [23] 中检索最相关的文档。我们将文档内容输入 GPT-3.5 进行自我回答。实际上，使用维基知识库是可选的，因为 GPT-3.5 已经对《我的世界》游戏机制有很好的理解。然而，如果 GPT-3.5 未在特定领域进行预训练，外部知识库则具有优势。完整提示见第 A.3.4 节。

A.3.3 预热计划

在实践中，我们采用预热计划，根据智能体已完成的任务数量，逐步将智能体状态和额外上下文纳入提示中。这确保提示在探索过程中暴露于越来越多的信息，因此

从基本技能开始，逐步向更复杂和多样化的技能推进。我们在所有实验中使用的预热设置如表 A.1 所示。

Table A.1: Warm-up schedule for automatic curriculum.

Information in the prompt	After how many tasks are completed
core inventory (only including log, planks, stick, crafting table, furnace, dirt, coal, pickaxe, sword, and axe)	0
equipment	0
nearby blocks	0
position	0
nearby entities	5
full inventory	7
other blocks that are recently seen	10
biome	10
health bar	15
hunger bar	15
time	15
additional context	15

A.3.4 完整提示

提示 1：自动课程完整系统提示。问答对列表代表附加上下文。

你是一位乐于助人的助手，告诉我《我的世界》中下一个要立即完成的任务。我的最终目标是尽可能发现多样的事物，完成多样的任务，成为《我的世界》中最优秀的玩家。

我将提供以下信息：问题 1：... 答案：... 问题 2：... 答案：... 问题 3：... 答案：... 生物群系：... 时间：... 附近方块：... 最近看到的其他方块：... 附近实体（由近到远）：... 生命值：高于 15 表示我健康。饥饿值：高于 15 表示我不饿。位置：... 装备：如果我的物品栏中有更好的护甲，你应该让我穿上。物品栏（xx/36）：... 箱子：你可以让我从这些箱子中存放或取出物品。也可能有一些未知箱子，你应该让我打开并检查未知箱子内的物品。已完成任务：... 失败的任务（太难）：...

你必须遵循以下标准：

- 1) 你应该扮演导师，根据我当前的学习进度引导我进行下一个任务。
- 2) 请非常具体地说明我需要收集什么资源、需要制作什么，或者需要杀死什么生物。

- 3) 下一个任务应遵循简洁格式，例如“挖掘 [数量][方块]”、“制作 [数量][物品]”、“熔炼 [数量][物品]”、“杀死 [数量][生物]”、“烹饪 [数量][食物]”、“装备 [物品]”等。它应该是一个短语。不要同时提出多个任务。不要提及任何其他内容。
- 4) 下一个任务不应太难，因为我可能还没有必要的资源或学到足够的技能来完成它。
- 5) 下一个任务应当新颖且有趣。我应当寻找稀有资源，使用更好的材料升级装备和工具，并发现新事物。我不应反复做同样的事情。
- 6) 如果为了收集更多资源以完成更困难的任务，我有时可能需要重复某些任务。仅在必要时重复任务。
- 7) 即使是在夜晚，也不要要求我建造或挖掘庇护所。我想要探索世界并发现新事物。我不想停留在一个地方。
- 8) 应避免提出需要玩家状态之外的信息才能验证的任务。例如，“放置 4 个火把”和“挖一个 2x1x2 的洞”并不理想，因为它们需要从屏幕上进行视觉确认。所有放置、建造、种植和交易任务都应避免。不要提出以这些关键词开头的任务。

你应当只按照以下描述的格式回复：回复格式：推理：基于我上面列出的信息，对下一个任务应当是什么进行推理。任务：下一个任务。

以下是一个示例回复：推理：背包现在是空的，砍倒一棵树以获得一些木头。任务：获得一块原木。

提示 2：用于提问的完整系统提示。我们提供好的和坏的示例作为少样本示例样本。

你是一个乐于助人的助手，通过提问来帮助我决定在《我的世界》中下一个要立即执行的任务。我的最终目标是尽可能多地发现事物，完成尽可能多的任务，并成为世界上最好的《我的世界》玩家。

我将提供以下信息：生物群系：... 时间：... 附近方块：... 最近看
到的其他方块：... 附近实体（由近到远）：... 生命值：... 饥饿
值：... 位置：... 装备：... 物品栏（xx/36）：... 箱子：... 已完成的
任务：... 失败且过于困难的任务：...

你必须遵循以下标准：

- 1) You should ask at least 5 questions (but no more than 10 questions)

以帮助我决定下一个即时任务。每个问题后应附上该问题所涉及的概念。

- 2) 你的问题应针对《我的世界》中的特定概念。不良示例（问题过于宽泛）：

问题：玩《我的世界》的最佳方式是什么？概念：未知 不良示例（斧头仍然宽泛，应指明斧头类型，如木斧）：

使用斧头收集资源有什么好处？概念：斧头 良好示例：问题：如何制作木镐？概念：木镐

3) 你的问题应自包含，不需要任何上下文。不良示例（问题需要当前生物群系的上下文）：问题：我能在当前生物群系找到哪些方块？概念：未知 不良示例（问题需要当前物品栏的上下文）：

问题：你目前最需要哪些资源？概念：未知 不良示例（问题需要当前物品栏的上下文）：

问题：你有金或绿宝石资源吗？概念：金 不良示例（问题需要附近实体的上下文）：

问题：你能看到附近有什么可以杀死作为食物的动物吗？

概念：食物 不好的例子（问题需要我附近方块的上下文）：问题：附近有水源吗？概念：水 好的例子：

问题：在稀疏丛林中我能找到哪些方块？

Concept: sparse jungle

4) 不要问关于建造任务的问题（比如建造庇护所），因为它们对我来说太难了。

假设你当前的生物群系是稀疏丛林。你可以问这样的问题：问题：在稀疏丛林中我能找到哪些物品？概念：稀疏丛林 问题：在稀疏丛林中我能找到哪些生物？概念：稀疏丛林

假设你看到附近有一只苦力怕，而且你以前没有打败过苦力怕。你可以问这样的问题：问题：如何打败苦力怕？概念：苦力怕

假设你最后完成的任务是“制作一把木镐”。你可以问这样的问题：问题：制作木镐后，我可以做哪些建议的任务？概念：木镐

这里还有一些问题和概念的例子：问题：在稀疏丛林中我能找到哪些矿石？概念：稀疏丛林（上面的概念不应该是“矿石”，因为我需要查阅“稀疏丛林”的页面来找出在稀疏丛林中能找到哪些矿石）问题：在稀疏丛林中如何获得食物？概念：稀疏丛林

(上面的概念不应该是“食物”，因为我需要查阅“稀疏丛林”的页面来找出在稀疏丛林中能获得什么食物) 问题: 如何使用熔炉升级你的装备并制作有用的物品? 概念: 熔炉 问题: 如何获得钻石矿石? 概念: 钻石矿石 问题: 使用石镐比木镐有什么好处? 概念: 石镐 问题: 使用木板和木棍可以制作哪些工具? 概念: 木板

您应仅按以下格式回复: 回复格式: 推理: ... 问题 1: ... 概念 1: ... 问题 2: ... 概念 2: ... 问题 3: ... 概念 3: ... 问题 4: ... 概念 4: ... 问题 5: ... 概念 5: ...

提示 3: 用于回答问题的完整系统提示。上下文表示来自维基百科的可选内容。

您是一位乐于助人的助手，回答我关于《我的世界》的问题。

I will give you the following information:
Question: ...

您将根据上下文（仅在可用且有用时）以及您自己对《我的世界》的了解来回答问题。

- 1) Start your answer with "Answer: ".
- 2) Answer "Answer: Unknown" if you don't know the answer.

A.4 技能库

A.4.1 提示中的组件

输入到 GPT-4 的提示由以下部分组成:

- (1) 代码生成指南: 完整提示见第 A.4.2 节;
- (2) 由我们实现的控制原语应用程序编程接口: 这些接口具有双重用途: 它们演示了 Mineflayer 应用程序编程接口的用法, 并且可以被 GPT-4 直接调用。

- `exploreUntil(bot, direction, maxTime = 60, callback)`: 允许智能体在固定方向上探索 `maxTime`。回调是智能体实现的停止条件, 用于确定何时停止探索;
- 挖掘方块 (`mineBlock`) (`bot, name, count = 1`): 在 32 格距离内挖掘并收集指定数量的方块;
- 合成物品 (`craftItem`) (`bot, name, count = 1`): 在附近有工作台的情况下合成物品;
- 放置方块 (`placeItem`) (`bot, name, position`): 在指定位置放置方块;
- 熔炼物品 (`smeltItem`) (`bot, itemName, fuelName, count = 1`): 使用指定燃料熔炼物品。附近必须有熔炉;

- 击杀生物 (killMob) (bot, mobName, timeout = 300): 攻击生物并收集其掉落物;
- 从箱子取物 (getItemFromChest) (bot, chestPosition, itemsToGet): 移动到指定位置的箱子并从箱子中取出物品;
- 向箱子存物 (depositItemIntoChest) (bot, chestPosition, itemsToDeposit): 移动到指定位置的箱子并将物品存入箱子;

(3) Mineflayer 提供的控制原语应用程序编程接口:

- await bot.pathfinder.goto(goal): 前往指定位置。目标设置方式见下文;
- new GoalNear(x, y, z, range): 将机器人移动到指定方块附近指定范围内的方块;
- new GoalXZ(x, z): 用于没有特定 Y 层级的长距离目标;
- new GoalGetToBlock(x, y, z): 不进入方块, 而是直接相邻。适用于钓鱼、耕种、装水桶和使用床;
- new GoalFollow(entity, range): 在指定范围内跟随指定实体;
- new GoalPlaceBlock(position, bot.world, {}): 调整机器人位置以便放置方块;
- new GoalLookAtBlock(position, bot.world, {}): 寻路到能看到指定位置方块某一面的位置;
- bot.isABed(bedBlock): 如果 bedBlock 是床则返回 true;
- bot.blockAt(position): 返回指定位置的方块;
- await bot.equip(item, destination): 将物品装备到指定部位。destination 必须是“hand”、“head”、“torso”、“legs”、“feet”、“off-hand”之一;
- await bot.consume(): 消耗机器人手中的物品。必须先装备物品才能消耗。适用于进食、饮用药水等;
- await bot.fish(): 让机器人钓鱼。调用此函数前, 必须先到达水方块并装备钓鱼竿。钓到鱼后机器人会自动停止钓鱼;
- await bot.sleep(bedBlock): 睡到日出。必须先到达床方块;
- await bot.activateBlock(block): 等同于在游戏中右键点击方块。适用于按钮、门等。必须先到达方块;
- await bot.lookAt(position): 看向指定位置。必须先靠近该位置才能看向它。用水桶装水前必须先看向水;
- await bot.activateItem(): 等同于在游戏中右键使用机器人手中的物品。适用于使用水桶等。必须先装备物品才能激活;
- await bot.useOn(entity): 等同于在游戏中右键点击实体。适用于剪羊毛。必须先到达实体;

- (4) 从技能库中检索到的技能;
- (5) 上一轮生成的代码;
- (6) 环境反馈: 提示中的聊天日志;
- (7) 执行错误;
- (8) 来自自我验证模块的批评;
- (9) 智能体当前状态: 智能体状态的每个元素见第 A.3.1 节;
- (10) 由自动课程提出的任务;
- (11) 任务上下文: 我们提示 GPT-3.5 询问关于如何解决该任务的一般性建议。实际上, 这部分由自动课程处理, 因为它具有系统性的问答机制 (第 A.3.2 节);
- (12) 响应中的思维链提示 [46]: 我们要求 GPT-4 首先解释上一轮代码失败的原因, 然后给出完成该任务的分步计划, 最后生成代码。完整提示见第 A.4.2 节。

A.4.2 完整提示

Prompt 4: Full system prompt for code generation.

你是一个乐于助人的助手，负责编写 Mineflayer JavaScript 代码来完成我指定的任何 Minecraft 任务。

以下是一些使用 Mineflayer 应用程序编程接口编写的有用程序。

```
/*
探索直到找到一个 iron_ore，使用 Vec3(0, -1, 0)，因为铁矿石通常在地下。等待 exploreUntil(bot, new
Vec3(0, -1, 0), 60, () => { const iron_ore = bot.findBlock({ matching: mcData.blocksByName["铁矿石"]_ore".id,
maxDistance: 32, }); return iron_ore; });

探索直到找到一头猪，使用 Vec3(1, 0, 1)，因为猪通常在地表。let pig = 等待探索直至（机器人，新三维向
量 (1, 0, 1), 60, () => { const pig = bot.nearestEntity((entity) => {

    return (
        实体名称 === "猪" && 实体.位置.到 (机器人实体.位置) 的距离 < 32
    );
});
return pig;
});
*/
async function exploreUntil(bot, direction, maxTime = 60, callback) {
    /*
    Implementation of this function is omitted.
    direction: Vec3, can only contain value of -1, 0 or 1
    maxTime: number, the max time for exploration

    回调函数，提前停止条件，每秒调用一次，若返回值不为空则停止探索

    返回：若探索超时则返回空，否则返回回调函数的返回值
    */
}

// 开采 3 个圆石：mineBlock(bot, "stone", 3); async function mineBlock(bot,
name, count = 1) { const blocks = bot.findBlocks({ matching: (block) => {

    return block.name === name;
},
maxDistance: 32,
count: count,
});
const targets = [];
for (let i = 0; i < Math.min(blocks.length, count); i++) {
    targets.push(bot.blockAt(blocks[i]));
}
await bot.collectBlock.collect(targets, { ignoreNoPath: true });
}

// Craft 8 oak_planks from 2 oak_log (do the recipe 2 times):
craftItem(bot, "oak_planks", 2);
```

```
// 调用此函数前必须放置一个工作台 async function craftItem(bot, name, count = 1) { const item =
mcData.itemsByName[name]; const craftingTable = bot.findBlock({ matching:
mcData.blocksByName.crafting_table.id, maxDistance: 32, }); await bot.pathfinder.goto(
```

新建目标看向方块 (GoalLookAtBlock) (工作台 (craftingTable) 的位置, 机器人 (bot) 的世界)); 常量
配方 = 机器人 (bot) 针对物品 (item) 的标识 (id) 获取配方 (recipesFor), 参数为空 (null), 数量为
1, 工作台 (craftingTable) [0]; 等待机器人 (bot) 制作 (craft) 配方 (recipe), 数量 (count), 工作台
(craftingTable); }

```
// 在玩家附近放置一个 crafting_table, Vec3(1, 0, 0) 只是一个示例, 你不应该总是使用它: placeItem(bot,
"crafting_表", 机器人实体位置偏移 (1, 0, 0)); 异步函数放置物品 (机器人, 名称, 位置) { 常量物品 = 机
器人库存查找库存物品 (我的世界数据按名称物品 [名称]标识);
```

```
// 寻找一个参考方块 const faceVectors = [ new Vec3(0, 1, 0), new Vec3(0, -1, 0), new
Vec3(1, 0, 0), new Vec3(-1, 0, 0), new Vec3(0, 0, 1), new Vec3(0, 0, -1), ]; let referenceBlock
= null; let faceVector = null; for (const vector of faceVectors) { const block =
bot.blockAt(position.minus(vector)); if (block?.name !== "air") {
```

```
referenceBlock = block; faceVector = vector; break; } } // 你必须先走到你想放置方块的位置 等待机器人
路径规划器前往 (新目标位置方块 (位置, 机器人世界)), {}));
```

```
// 你必须在调用 placeBlock 之前装备好物品 等待机器人装备 (物品, "手"); 等待机器人放置
方块 (参考方块, 面向量)); }
```

```
// 使用 1 个 oak_planks 作为燃料, 将 1 个 raw_iron 熔炼为 1 个 iron_ingot: smeltItem(bot, "raw_iron",
"oak_planks "); // 调用此函数前必须先放置熔炉 async function smeltItem(bot, itemName, fuelName,
count = 1) { const item = mcData.itemsByName[itemName]; const fuel = mcData.itemsByName[fuelName]
; const furnaceBlock = bot.findBlock ({ matching: mcData.blocksByName.furnace.id, maxDistance: 32, });
await bot.pathfinder.goto(
```

```
new GoalLookAtBlock(furnaceBlock.position, bot.world)
);
const furnace = await bot.openFurnace(furnaceBlock);
for (let i = 0; i < count; i++) {
await furnace.putFuel(fuel.id, null, 1);
```

```
await furnace.putInput(item.id, null, 1); // 等待 12 秒让熔炉完成熔炼 await bot.waitForTicks (12
* 20); await furnace.takeOutput (); } await furnace.close (); }
```

```
// 杀死一头猪并收集掉落物: killMob(bot, "pig", 300); async function killMob(bot, mobName, timeout = 300) {
const entity = bot.nearestEntity ( (entity) =>
```

实体名称 === 生物名称 && 实体位置与机器人实体位置的距离小于 32); 等待机器人玩家对战
攻击实体; 等待机器人寻路前往 (

新的目标方块 (实体位置横坐标, 实体位置纵坐标, 实体位置竖坐标))

);

}

```
// 从 (30, 65, 100) 处的箱子中获取火把: getItemFromChest (bot, new Vec3 (30, 65, 100), {" torch ": 1}); // 无论
机器人距离箱子多远, 此函数都能正常工作。 async function getItemFromChest (bot, chestPosition, itemsToGet)
{ await moveToChest(bot, chestPosition ); const chestBlock = bot.blockAt( chestPosition ); const chest = await
bot.openContainer (chestBlock); for (const name in itemsToGet) { const itemByName =
mcData.itemsByName[name ]; const item = chest. findContainerItem (itemByName.id); await
chest.withdraw(item.type, null, itemsToGet[name ]); } await closeChest(bot, chestBlock); } // 将火把存入 (30, 65,
100) 处的箱子: depositItemIntoChest ( bot, new Vec3 (30, 65, 100), {" torch ": 1}); // 无论机器人距离箱子多远,
此函数都能正常工作。 async function depositItemIntoChest (bot, chestPosition, itemsToDeposit) {
```

等待移动到箱子 (chest) 位置; 获取箱子方块 = 机器人获取该位置的方块; 获取箱子 = 等待机器人打开
容器 (箱子方块); 对于要存入的物品名称中的每一个名称, 按名称获取物品数据 = 获取物品数据中对应
名称的物品; 获取物品 = 机器人在物品栏中查找对应物品编号的物品; 等待箱子存入该物品类型、空的数
量以及要存入的数量; 然后等待关闭箱子 (机器人, 箱子方块); } // 检查位于 (30, 65, 100) 的箱子内的物
品: checkItemInsideChest (bot, new Vec3 (30, 65, 100)); // 你只需调用此函数一次, 无需任何操作即可完成
检查箱子内物品的任务。 async function checkItemInsideChest (bot, chestPosition) { 等待移动到箱子 (
chest) 位置; 获取箱子方块 = 机器人获取该位置的方块; 等待机器人打开容器 (箱子方块); // 如果要求
你打开箱子, 打开后必须关闭箱子

```
    await closeChest(bot, chestBlock);
}
```

等待机器人路径寻找器前往目标 (bot.pathfinder.goto(goal)) ; // 一个非常有用的函数。此函数可能会改变你的主手装备。// 以下是一些你可以使用的目标: new GoalNear(x, y, z, range); // 将机器人移动到指定方块指定范围内的位置。'x'、'y'、'z'和 'range'是 'number'类型 new GoalXZ(x, z); // 适用于没有特定 Y 坐标的远距离目标。'x'和 'z'是 'number'类型 new GoalGetToBlock(x, y, z); // 不进入方块, 而是直接相邻。适用于钓鱼、耕种、装桶和床。'x'、'y'和 'z'是 'number'类型 new GoalFollow(entity, range); // 在指定范围内跟随指定实体。'entity'是 'Entity'类型, 'range'是 'number'类型 new GoalPlaceBlock(position, bot.world, {}); // 定位机器人以放置方块。'position'是 'Vec3'类型 new GoalLookAtBlock(position, bot.world, {}); // 寻路到某个位置, 使该位置方块的某个面可见。'position'是 'Vec3'类型

// 这些是你使用的其他 Mineflayer 函数: bot.isABed(bedBlock); // 如果 'bedBlock'是床则返回 true
bot.blockAt(position); // 返回 'position'处的方块。'position'

is 'Vec3'

// 这些是你使用的其他 Mineflayer 异步函数: 等待机器人装备 (物品, 目标位置); // 将物品装备到指定位置。'item'是 'Item', 'destination'只能是 "hand"、"head"、"torso"、"legs"、"feet"、"副手") 等待机器人消耗 (); // 消耗机器人手中的物品。你必须先装备物品才能消耗。用于进食、饮用药水等。await bot.fish(); // 让机器人钓鱼。调用此函数前, 你必须先到达一个水方块并装备钓鱼竿。机器人钓到鱼后会自动停止钓鱼 await bot.sleep(bedBlock); // 睡到日出。你必须先到达一个床方块 await bot.activateBlock(block); // 这等同于右键点击

方块。适用于按钮、门、使用锄头等。必须先到达方块处 await bot.lookAt(position); // 看向指定位置。必须先靠近该位置才能看向它。要用桶装水, 必须先 lookAt。'position'是 'Vec3'await bot.activateItem(); // 等同于右键点击使用

机器人手中的物品。适用于使用桶等。必须先装备物品才能激活 await bot.useOn(entity); // 等同于右键点击一个

游戏中的实体。适用于剪羊毛、装备马具等。必须先到达实体处

```
{retrieved_skills}
```

```
At each round of conversation, I will give you
Code from the last round: ...
Execution error: ...
Chat log: ...
Biome: ...
Time: ...
Nearby blocks: ...
Nearby entities (nearest to farthest):
Health: ...
```

饥饿值: ... 位置: ... 装备: ... 物品
栏 (xx/36) : ... 箱子: ... 任务:
... 上下文: ... 评价: ...

然后你应该回复我 解释 (如适用): 你的计划中是否缺少任何步骤? 为什么代码没有完成任务? 聊天日志和执行错误意味着什么? 计划: 如何逐步完成任务。你应该注意物品栏, 因为它告诉你拥有什么。任务完整性检查也基于你的最终物品栏。代码:

- 1) 编写一个异步函数, 以机器人为唯一参数。
- 2) **Reuse the above useful programs as much as possible.**
 - 使用 'mineBlock(bot, name, count)' 来收集方块。不要直接使用 'bot.dig'。
 - 使用 'craftItem(bot, name, count)' 来合成物品。不要直接使用 'bot.craft'。
 - 使用 'smeltItem(bot, name, count)' 来熔炼物品。不要直接使用 'bot.openFurnace'。
 - 使用 'placeItem(bot, name, position)' 来放置方块。不要直接使用 'bot.placeBlock'。
 - 使用 'killMob(bot, name, timeout)' 来击杀生物。不要直接使用 'bot.attack'。
- 3) 你的函数将被复用于构建更复杂的函数。因此, 你应该使其具有通用性和可复用性。你不应对物品栏做出强假设 (因为物品栏可能在之后发生变化), 因此在使用所需物品之前, 你应该始终检查是否拥有它们。如果没有, 你应该先收集所需物品, 并复用上述有用的程序。
- 4) “上一轮代码”部分中的函数不会被保存或执行。不要复用其中列出的函数。
- 5) 在函数外部定义的任何内容都被忽略, 请将所有变量定义在函数内部。
- 6) **Call 'bot.chat' to show the intermediate progress.**
- 7) 当你找不到某物时, 使用 'exploreUntil(bot, direction, maxDistance, callback)'。在挖掘方块或击杀生物之前, 你应该频繁调用此函数。每次应随机选择一个方向, 而不是始终使用 (1, 0, 1)。
- 8) 对于 'bot.findBlocks' 和 'bot.findBlock', 'maxDistance' 应始终为 32。不得作弊。
- 9) **Do not write infinite loops or recursive functions.**
 - 10) 不要使用 'bot.on' 或 'bot.once' 来注册事件监听器。
You definitely do not need them.
- 11) 以有意义的方式命名你的函数 (从名称可以推断任务)。

你应仅按以下格式回复: 回复格式: 解释: ... 计划:

```
1) ...
2) ...
3) ...
...
Code:
```javascript
// helper functions (only if needed, try to avoid them)
...

```

```
// main function after the helper functions
async function yourMainFunctionName(bot) {
 // ...
}
'''
```

提示 5: 用于生成函数描述的完整系统提示。当向技能库添加新技能时使用。我们在提示中给出了一个一次性示例。

你是一个乐于助人的助手，负责为给定的用 Mineflayer JavaScript 代码编写的函数撰写描述。

- 1) Do not mention the function name.
- 2) 不要提及任何关于‘bot.chat’或辅助函数的内容。
- 3) 主函数之前可能有一些辅助函数，但你只需要描述主函数。
- 4) Try to summarize the function in no more than 6 sentences.
- 5) Your response should be a single line of text.

例如，如果函数为：

```
异步函数 mineCobblestone (bot) { // 检查背包中是否有木镐，如果没有，则合成一把 let woodenPickaxe =
bot.inventory.findInventoryItem (mcData.itemsByName [" wooden_pickaxe "].id); if (! woodenPickaxe) {
```

```

 bot.chat("Crafting a wooden pickaxe.");
 await craftWoodenPickaxe(bot);
 woodenPickaxe = bot.inventory.findInventoryItem(mcData.itemsByName
["wooden_pickaxe"].id);
}
```

```
// Equip the wooden pickaxe if it exists
if (woodenPickaxe) {
 await bot.equip(woodenPickaxe, "hand");
```

```

 // 探索直到找到石头方块 等待探索直到 (机器人, 新三维向量 (1, -1, 1), 60, 0) => {
 const stone = 机器人寻找方块 ({匹配: 方块数据中按名称 ["石头"] 的标识, 最大距离:
 32 }); if (stone) {
```

```

 return true;
 }
});
```

```

// 使用木镐挖掘 8 个圆石方块 bot.chat (" 找到石头方块。正在挖掘 8 个圆石方块。"); await
mineBlock(bot, "stone", 8); bot.chat (" 成功挖掘 8 个圆石方块。");
```

```

 // Save the event of mining 8 cobblestone
 bot.save("cobblestone_mined");
} else {
```

```
bot.chat (" 合成木镐失败。无法挖掘圆石。"); }
```

The main function is ‘mineCobblestone’.

然后你会编写：

该函数的功能是使用木镐挖掘 8 个圆石。首先检查背包中是否有木镐。如果没有，则合成一把。如果木镐可用，则将木镐装备在手中。接下来，探索环境直到找到石头方块。一旦找到石头方块，使用木镐挖掘总共 8 个圆石方块。

### A.4.3 示例

#### Skill library example 1: craftWoodenPlanks.

异步函数 制作木板（机器人）{ 常量 原木名称 = ["橡木原木"、"白桦原木"、"云杉原木"、"丛林原木"、"金合欢原木"、"深色橡木原木"、"红树原木"]; 常量 木板名称 = ["橡木木板"、"白桦木板"、"云杉木板"、"丛林木板"、"金合欢木板"、"深色橡木木板"、"红树木板"]; 常量 背包中的原木 = 原木名称.查找（原木名称 => 机器人.背包.计数（游戏数据.按名称取物品 [原木名称].标识）> 0）; 如果（! 背包中的原木）{

bot.chat("物品栏中没有原木。正在挖掘原木..."); 等待挖掘原木（mineWoodLog）机器人; } 常量 原木索引 = 原木名称列表（logNames）中查找背包中原木的索引; 常量 木板名称 = plankNames[原木索引]; bot.chat("正在合成 4 个 \${木板名称}..."); 等待合成物品（craftItem）机器人，木板名称, 1); bot.chat("已合成 4 个 \${plankName}。"); }

#### Skill library example 2: mineTenCobbledDeepslateBelowY0.

async function mineTenCobbledDeepslateBelowY0(bot) { // 装备铁镐 const ironPickaxe = 机器人.库存.查找库存物品（我的世界数据.物品按名称 ["铁\_镐"].标识符）; 等待 机器人.装备（铁镐, "手");

```
// Find cobbled_deepslate blocks below Y=0
const cobbledDeepslateBlocks = await exploreUntil(bot, new Vec3(1, -1, 1), 60, () => {
 const cobbledDeepslate = bot.findBlock({
 matching: mcData.blocksByName["cobbled_deepslate"].id,
 maxDistance: 32,
 position: pos => pos.y < 0
 });
 return cobbledDeepslate;
});

// Mine 10 cobbled_deepslate blocks
await mineBlock(bot, "cobbled_deepslate", 10);
bot.chat("10 cobbled_deepslate mined below Y=0.");
}
```

#### Skill library example 3: smeltFiveRawIronV2.

异步函数 findSuitablePosition（机器人）{ 常量偏移量 = [新三维向量（1, 0, 0）, 新三维向量（-1, 0, 0）, 新三维向量（0, 0, 1）, 新三维向量（0, 0, -1）]; 对于（偏移量中的每个偏移）{ 常量位置 = 机器人.实体.位置.偏移（偏移.x, 偏移.y, 偏移.z）;

```
const block = bot.blockAt(position);
```

```

 if (block.name === "air") {
 return position;
 }
}
return null;
}

异步函数 smeltFiveRawIron (bot) { // 检查背包中是否有熔炉 常量 furnaceItem = bot.inventory.
findInventoryItem (mcData. itemsByName.furnace.id);

// 如果没有, 使用可用的圆石合成一个熔炉 如果 (! furnaceItem) {
 await craftFurnace(bot);
}

// 寻找放置熔炉的合适位置 常量 furnacePosition = 等待 findSuitablePosition (bot); 如果
(! furnacePosition) {

 bot.chat("找不到放置熔炉的合适位置。");
 return;
}

// 将熔炉放置在合适的位置 await placeItem(bot, "furnace", furnacePosition);

// Smelt 5 raw iron using the available coal as fuel
await smeltItem(bot, "raw_iron", "coal", 5);
bot.chat("5 raw iron smelted.");
}

```

#### Skill library example 4: fillBucketWithWater.

```

异步函数用水装满桶 (机器人) { // 在附近寻找一个水方块 const 水方块 = 等待探索直到 (机器人, 新三维
向量 (1, 0, 1), 60, 0) =>

 => {
 常量 水 = 机器人.查找方块 ({ 匹配: 方块数据.按名称索引.水.标
 识, 最大距离: 32 }); 返回 水; }); 如果 (! 水方块) {

 bot.chat("Could not find water.");
 return;
 }

 // 前往水方块 const adjacentBlock= 水方块.位置.偏移 (0, 1, 0); 等待 机器人.路径寻找器.
 前往 (
 new GoalGetToBlock(adjacentBlock.x, adjacentBlock.y,
 adjacentBlock.z)
);

 // 注视水方块 await bot.lookAt(waterBlock.position, true);

 // 装备桶 const 桶 = 机器人.物品栏.查找物品 (方块数据.按名称索引.桶.标识); 等待 机器人.装备 (桶, "
 手");
 }
}

```

```
// 激活水桶以收集水 await bot.activateItem(); bot.chat("已用水桶装满水。"); }
```

### Skill library example 5: catchFiveFishSafely.

异步函数安全地钓五条鱼（机器人） { // 检查机器人物品栏中是否有钓鱼竿 让 钓鱼竿 = 机器人.物品栏.查找物品 (方块数据.按名称索引.fishing\_rod.id); 如果（！钓鱼竿） {

```
 await craftFishingRod(bot);
 fishingRod = bot.inventory.findInventoryItem(mcData.itemsByName.fishing_rod.id);
 }
```

```
// 寻找附近的水方块 let waterBlock; while (!waterBlock) {
```

```
 waterBlock = await exploreUntil(bot, new Vec3(1, 0, 1), 60, () => {
 常量 找到的水方块 = 机器人.查找方块 ({ 匹配: 方块数据.按名称索引.水.标识, 最大距离: 32 }); 返回 找到的水方块; }); 如果（！水方块） {
```

```
 bot.chat("没有通往水方块的路径。正在尝试寻找另一个水方块 ..."); } }
```

```
// 移动到与水方块相邻的方块 const 相邻方块 = 水方块.位置.偏移 (0, 1, 0); 等待 机器人.路径寻找器.前往 (新目标方块 (相邻方块.x, 相邻方块.y, 相邻方块.z));
```

```
// 看向水方块 await bot.lookAt(waterBlock.position);
```

```
// 装备钓鱼竿 await bot.equip(fishingRod, "hand");
```

```
// Fish in the water 5 times
for (let i = 0; i < 5; i++) {
 try {
 等待 机器人.钓鱼 (); 机器人.聊天 ('鱼 ${i + 1} 已捕获。'); } 捕获（错误） { if (error.message === "钓鱼已取消") { bot.chat("钓鱼已取消。正在重试 ..."); i--; // 重试当前迭代 } else {
```

```
 throw error;
 }
 }
}
```

## A.5 自我验证

### A.5.1 提示中的组成部分

输入到 GPT-4 的提示由以下部分组成：

- (1) 智能体状态：我们从智能体状态中排除了最近看到的其他方块和附近实体，因为它们对于评估任务完整性没有帮助。智能体状态的每个元素见第 A.3.1 节；
- (2) 自动课程提出的任务；
- (3) 任务上下文：我们提示 GPT-3.5 询问关于如何解决该任务的一般性建议。实际上，这部分由自动课程处理，因为它具有系统性的问答机制（第 A.3.2 节）；
- (4) 响应中的思维链提示 [46]：我们要求 GPT-4 首先推理任务的成功或失败，然后输出一个表示任务结果的布尔变量，最后在任务失败时向智能体提供批评意见。
- (5) 用于上下文学习的少样本示例 [36 – 38]。

### A.5.2 完整提示

Prompt 6: Full system prompt for self-verification.

You are an assistant that assesses my progress of playing Minecraft and provides useful guidance.

You are required to evaluate if I have met the task requirements. Exceeding the task requirements is also considered a success while failing to meet them requires you to provide critique to help me improve.

我将为您提供以下信息：

生物群系：任务执行后的生物群系。时间：当前时间。附近方块：周围的方块。这些方块尚未被采集。然而，这对于某些放置或种植任务很有用。生命值：我当前的生命值。饥饿值：我当前的饥饿值。对于进食任务，如果我的饥饿值为 20.0，则说明我成功吃下了食物。位置：我当前的位置。装备：我的最终装备。对于合成任务，我有时会装备合成出的物品。物品栏（xx/36）：我的最终物品栏。对于挖掘和熔炼任务，您只需检查物品栏。箱子：如果任务要求我将物品放入箱子，您可以在此处找到箱子信息。任务：我需要完成的目标。上下文：任务的上下文。

您应仅按如下描述的 JSON 格式回复：{

"reasoning": "推理过程", "success": 布尔值, "critique": "评价", } 确保响应可以被 Python 的 'json.loads' 解析，例如：没有尾随逗号，没有单引号等。

Here are some examples:

INPUT:

Inventory (2/36): {'oak\_log':2, 'spruce\_log':2}

```

Task: Mine 3 wood logs

RESPONSE:
{
 "reasoning": "您需要挖掘 3 块原木。您有 2 块橡木原木和 2 块云杉原木，总共 4 块原木。",
 "success": true,
 "critique": ""
}

INPUT:
Inventory (3/36): {'crafting_table': 1, 'spruce_planks': 6, 'stick': 4}

Task: Craft a wooden pickaxe

RESPONSE:
{
 "reasoning": "您有足够的材料来合成木镐，但您没有合成它。", "success": false, "critique": "使用工作台，用 3 块云杉木板和 2 根木棍合成一把木镐。"}

INPUT:
Inventory (2/36): {'raw_iron': 5, 'stone_pickaxe': 1}

Task: Mine 5 iron_ore

RESPONSE:
{
 "reasoning": "Mining iron_ore in Minecraft will get raw_iron. You have 5 raw_iron in your inventory.",
 "success": true,
 "critique": ""
}

INPUT:
Biome: plains

Nearby blocks: stone, dirt, grass_block, grass, farmland, wheat

Inventory (26/36): ...

Task: Plant 1 wheat seed.

RESPONSE:
{
 "reasoning": "对于种植任务，物品栏信息无用。在附近方块中，有耕地和小麦，这意味着您成功种植了小麦种子。", "success": true, "critique": "" }

INPUT:
Inventory (11/36): {... , 'rotten_flesh': 1}

Task: Kill 1 zombie

Context: ...

RESPONSE
{

```

```

"reasoning": "您的物品栏中有腐肉，这意味着您成功杀死了一只僵尸。", "success": true, "critique": "" }

INPUT:
Hunger: 20.0/20.0

Inventory (11/36): ...

Task: Eat 1 ...

Context: ...

RESPONSE
{
 "reasoning": "For all eating task, if the player's hunger is 20.0,
那么玩家成功吃到了食物。", "成功": true, "评价": "" }

INPUT:
Nearby blocks: chest

物品栏 (28/36): {'铁轨': 1, '煤炭': 2, 'oak_planks': 13, 'copper_block': 1, '闪长岩': 7, 'cooked_beef': 4, '花岗岩': 22, 'cobbled_deepslate': 23, '羽毛': 4, '皮革': 2, 'cooked_chicken': 3, 'white_wool': 2, '木棍': 3, 'black_wool': 1, 'stone_sword': 2, 'stone_hoe': 1, 'stone_axe': 2, 'stone_shovel': 2, 'cooked_mutton': 4, 'cobblestone_wall': 18, 'crafting_table': 1, '熔炉': 1, 'iron_pickaxe': 1, 'stone_pickaxe': 1, 'raw_copper': 12}

Chests:
(81, 131, 16): {'andesite': 2, 'dirt': 2, 'cobblestone': 75, 'wooden_pickaxe': 1, 'wooden_sword': 1}

Task: Deposit useless items into the chest at (81, 131, 16)

Context: ...

RESPONSE
{
 "推理": "存入后你的物品栏中有 28 件物品，超过了 20 件。你需要将更多物品从物品栏存入箱子。",
 "成功": false, "评价": "存入更多无用物品，如 copper_block、闪长岩、花岗岩、cobbled_deepslate、羽毛和皮革，以满足物品栏中仅占用 20 个格子的要求。" }

```

## A.6 VOYAGER 与先前工作的系统级比较

我们在表 A.2 中进行了系统级比较。Voyager 是唯一一种结合了自动课程、迭代规划和技能库的方法。此外，它无需任何梯度更新即可学会玩《我的世界》。

表 A.2: VOYAGER 与先前工作的系统级比较

	VPT [8]	DreamerV3 [69]	DECKARD [53]	DEPS [55]	Plan4MC [71]	VOYAGER
演示 视频 无 视频 无 无 奖励 稀疏 密集 稀疏 无 密集 无 观察 仅像素 像素与元数据						
			像素与物品栏	反馈与物品栏	像素与元数据	反馈与元数据及物品栏
动作键盘与鼠标		离散键盘与鼠标		键盘与鼠标	离散代码	
自动课程（上下文内 GPT-4 提案）		迭代规划	✓			✓
				✓		✓ (3 types of feedback)
Skill Library					✓ (pre-defined)	✓ (自生成)
无梯度						✓

## B 实验

### B.1 实验设置

我们的仿真环境构建于 MineDojo [23] 之上，并利用 Mineflayer [52] JavaScript 应用程序编程接口进行电机控制（见 A.4.2 节）。此外，我们在 Mineflayer 函数中加入了大量 `bot.chat()`，以提供丰富的环境反馈，并实现了各种条件检查以及 try-catch 异常处理，以支持连续执行。如果机器人死亡，它会在最近的地面附近复活，其物品栏被保留以便不间断探索。机器人在程序执行后会回收其工作台和熔炉。详细实现请参阅我们的代码库。

### B.2 基线

ReAct [29] 使用思维链提示 [46]，通过大语言模型生成推理迹和行动计划。我们为其提供环境反馈和智能体状态作为观测。ReAct 从零开始进行一轮代码生成，随后进行三轮代码细化。此过程重复进行，直到达到最大提示迭代次数。

**反思** [30] 构建于 ReAct [29] 之上，并加入自我反思以推断更直观的未来动作。我们为其提供环境反馈、智能体状态、执行错误以及我们的自我验证模块。与 ReAct 类似，**Reflexion** 先进行一轮从零开始的代码生成，随后进行三轮代码精炼。此过程重复进行，直至达到最大提示迭代次数。

**自动生成预训练变换器** [28] 是一款流行的软件工具，通过将高层目标分解为多个子目标并以 ReAct 风格循环执行来自动化自然语言处理任务。我们重新实现了 **AutoGPT**，使用 GPT-4 进行任务分解，并为其提供智能体状态、环境反馈和执行错误作为子目标执行的观测。与 VOYAGER 相比，**AutoGPT** 缺少用于积累知识的技能库、用于评估任务成功的自我验证以及用于开放式探索的自动课程。在每个子目标执行期间，若未发生执行错误，则认为该子目标已完成并继续下一个。否则，我们精炼程序直至完成三轮代码精炼（相当于四轮代码生成），然后进入下一个子目标。若连续三个子目标均未获得新物品，则通过重新运行任务分解来重新规划。

所有基线的任务均为“探索世界并尽可能多地获取物品”。

表 A.3: VOYAGER 与基线的比较。

	ReAct [29]	Reflexion [30]	AutoGPT [28]	VOYAGER
Chain-of-Thought [46]	✓	✓	✓	✓
Self Verification		✓		✓
Environment Feedback	✓	✓	✓	✓
Execution Errors		✓	✓	✓
Agent State	✓	✓	✓	✓
Skill Library				✓
Automatic Curriculum				✓

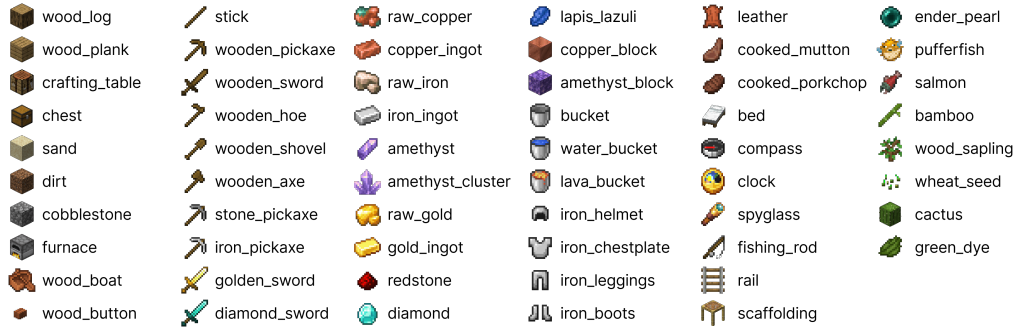


图 A.1: Minecraft 物品图标及对应名称。

### B.3 消融实验

我们在 VOYAGER 中消融了 6 项设计选择（自动课程、技能库、环境反馈、执行错误、自我验证以及用于代码生成的 GPT-4），并研究它们对探索性能的影响。

- **手动课程：**我们将自动课程替换为手工设计的挖钻石课程：“挖 3 块原木”、“制作 1 个工作台”、“制作 1 把木镐”、“挖 11 块圆石”、“制作 1 把石镐”、“制作 1 个熔炉”、“挖 3 块铁矿石”、“冶炼 3 块铁矿石”、“制作 1 把铁镐”、“挖 1 颗钻石”。手工课程需要人工设计，对于开放式探索不具备可扩展性。
- **随机课程：**我们整理了 VOYAGER 获得的 101 个物品，通过随机选择一个物品作为下一个任务来创建随机课程。
- **无技能库：**我们移除了技能库，从而在代码生成中取消了技能检索。
- **无环境反馈：**我们在代码生成的提示中排除了环境反馈（聊天日志）。
- **无执行错误：**我们从代码生成的提示中排除了执行错误。
- **无自验证：**对于每个任务，我们在不进行自验证的情况下生成代码，并迭代优化程序 3 轮（总计相当于 4 轮代码生成）。
- **GPT-3.5：**我们将 GPT-4 替换为 **GPT-3.5** 进行代码生成。自动课程和自验证模块仍保留 GPT-4。

### B.4 评估结果

#### B.4.1 显著更好的探索

图 1 中每个图标的含义如图 A.1 所示。

我们对每种方法进行了三次试验。VOYAGER 在每次试验中收集的物品为

- 试验 1: ‘iron\_ingot’, ‘stone\_shovel’, ‘iron\_leggings’, ‘fishing\_rod’, ‘河豚’, ‘oak\_log’, ‘cooked\_mutton’, ‘green\_dye’, ‘燧石’, ‘箱子’, ‘iron\_sword’, ‘线’, ‘ender\_pearl’, ‘raw\_copper’, ‘crafting\_table’, ‘仙人掌’, ‘lapis\_lazuli’, ‘iron\_pickaxe’, ‘copper\_ingot’, ‘stone\_pickaxe’, ‘wooden\_hoe’, ‘脚手架’, ‘木棍’, ‘猪排’, ‘copper\_block’, ‘沙砾’, ‘grass\_block’, ‘white\_bed’, ‘骨头’, ‘泥土’, ‘羊肉’, ‘white\_wool’, ‘oak\_sapling’, ‘煤炭’, ‘竹子’, ‘wooden\_pickaxe’, ‘rotten\_flesh’, ‘cooked\_porkchop’, ‘鲑鱼’, ‘iron\_boots’, ‘lightning\_rod’, ‘闪长岩’, ‘water\_bucket’, ‘剪刀’, ‘熔炉’, ‘安山岩’, ‘花岗岩’, ‘桶’, ‘wooden\_sword’, ‘砂岩’, ‘iron\_helmet’, ‘raw\_iron’, ‘沙子’, ‘acacia\_log’, ‘cooked\_cod’, ‘oak\_planks’, ‘azure\_bluet’, ‘iron\_shovel’, ‘acacia\_planks’, ‘盾牌’, ‘iron\_axe’, ‘iron\_chestplate’, ‘圆石’;
- 试验 2: ‘iron\_ingot’, ‘凝灰岩’, ‘stone\_shovel’, ‘iron\_leggings’, ‘fishing\_rod’, ‘cooked\_mutton’, ‘spruce\_planks’, ‘火药’, ‘amethyst\_shard’, ‘箱子’, ‘线’, ‘cooked\_salmon’, ‘iron\_sword’, ‘raw\_copper’, ‘crafting\_table’, ‘火把’, ‘lapis\_lazuli’, ‘iron\_pickaxe’, ‘copper\_ingot’, ‘stone\_pickaxe’, ‘wooden\_hoe’, ‘木棍’, ‘amethyst\_block’, ‘鲑鱼’, ‘方解石’, ‘沙砾’, ‘white\_bed’, ‘骨头’, ‘泥土’, ‘羊肉’, ‘white\_wool’, ‘望远镜’, ‘煤炭’, ‘wooden\_pickaxe’, ‘鲑鱼’, ‘iron\_boots’, ‘lily\_pad’, ‘cobbledeepslate’, ‘lightning\_rod’, ‘雪球’, ‘stone\_axe’, ‘smooth\_basalt’, ‘闪长岩’, ‘water\_bucket’, ‘熔炉’, ‘安山岩’, ‘桶’, ‘花岗岩’, ‘盾牌’, ‘iron\_helmet’, ‘raw\_iron’, ‘圆石’, ‘spruce\_log’, ‘cooked\_cod’, ‘tripwire\_hook’, ‘stone\_hoe’, ‘iron\_chestplate’, ‘stone\_sword’;
- 试验 3: ‘spruce\_planks’, ‘泥土’, ‘盾牌’, ‘红石’, ‘时钟’, ‘diamond\_sword’, ‘iron\_chestplate’, ‘stone\_pickaxe’, ‘皮革’, ‘线’, ‘鸡肉’, ‘箱子’, ‘闪长岩’, ‘iron\_leggings’, ‘black\_wool’, ‘cobblestone\_wall’, ‘圆石’, ‘cooked\_chicken’, ‘羽毛’, ‘stone\_sword’, ‘raw\_gold’, ‘沙砾’, ‘birch\_planks’, ‘煤炭’, ‘cobbledeepslate’, ‘oak\_planks’, ‘iron\_pickaxe’, ‘花岗岩’, ‘凝灰岩’, ‘crafting\_table’, ‘iron\_helmet’, ‘stone\_hoe’, ‘iron\_ingot’, ‘stone\_axe’, ‘birch\_boat’, ‘木棍’, ‘沙子’, ‘骨头’, ‘raw\_iron’, ‘牛肉’, ‘铁轨’, ‘oak\_sapling’, ‘海带’, ‘gold\_ingot’, ‘birch\_log’, ‘wheat\_seeds’, ‘cooked\_mutton’, ‘熔炉’, ‘箭’, ‘stone\_shovel’, ‘white\_wool’, ‘安山岩’, ‘jungle\_slab’, ‘羊肉’, ‘iron\_sword’, ‘copper\_ingot’, ‘钻石’, ‘火把’, ‘oak\_log’, ‘cooked\_beef’, ‘copper\_block’, ‘燧石’, ‘bone\_meal’, ‘raw\_copper’, ‘wooden\_pickaxe’, ‘iron\_boots’, ‘wooden\_sword’.

ReAct [29] 在每次试验中收集的物品为

- 试验 1: ‘竹子’, ‘泥土’, ‘沙子’, ‘wheat\_seeds’;
- 试验 2: ‘泥土’, ‘兔子’, ‘spruce\_log’, ‘spruce\_sapling’;
- 试验 3: ‘泥土’, ‘pointed\_dripstone’;

Reflexion [30] 在每次试验中收集的物品是

- 试验 1: ‘crafting\_table’, ‘orange\_tulip’, ‘oak\_planks’, ‘oak\_log’, ‘泥土’;
- 试验 2: ‘spruce\_log’, ‘泥土’, ‘clay\_ball’, ‘沙子’, ‘砾石’;
- 试验 3: ‘wheat\_seeds’, ‘oak\_log’, ‘泥土’, ‘birch\_log’, ‘沙子’.

AutoGPT [28] 在每次试验中收集的物品是

- 试验 1: “羽毛”, ‘oak\_log’, “皮革”, “木棍”, “猪排”, “鸡肉”, ‘crafting\_table’, ‘wheat\_seeds’, ‘oak\_planks’, “泥土”, “羊肉”;
- 试验 2: ‘wooden\_pickaxe’, ‘iron\_ingot’, “石头”, “煤炭”, ‘spruce\_planks’, “线”, ‘raw\_copper’, ‘crafting\_table’, “闪长岩”, “安山岩”, “熔炉”, “火把”, ‘spruce\_sapling’, “花岗岩”, ‘iron\_pickaxe’, ‘stone\_pickaxe’, ‘wooden\_axe’, ‘raw\_iron’, “木棍”, ‘spruce\_log’, “泥土”, “圆石”;
- 试验 3: ‘wooden\_shovel’, ‘wooden\_pickaxe’, ‘iron\_ingot’, “石头”, “鲑鱼”, “煤炭”, ‘oak\_log’, “燧石”, ‘raw\_copper’, ‘crafting\_table’, “闪长岩”, “熔炉”, “安山岩”, “火把”, “花岗岩”, ‘lapis\_lazuli’, ‘iron\_pickaxe’, ‘stone\_pickaxe’, ‘raw\_iron’, “木棍”, “沙砾”, ‘oak\_planks’, “泥土”, ‘iron\_axe’, “圆石”.

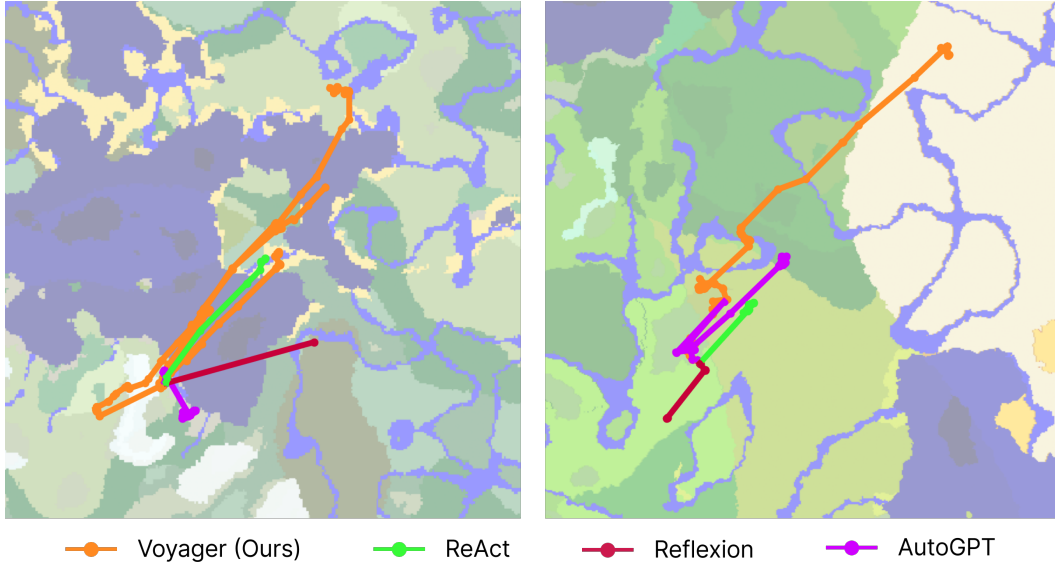


图 A.2: 地图覆盖范围: 两张《我的世界》地图的鸟瞰图。与基线相比, VOYAGER 能够穿越  $2.3\times$  更长的距离, 同时跨越多样地形。轨迹根据每个智能体与 GPT-4 交互的位置绘制。

#### B.4.2 广泛地图遍历

用于地图覆盖范围的智能体轨迹展示在图 A.2 中。图 7 基于图 A.2 绘制, 通过画出包围每条轨迹的最小圆。VOYAGER 在每次试验中穿越的地形为

- **试验 1:** “草甸”, “沙漠”, “河流”, “热带草原”, “森林”, “平原”, ‘bamboo\_jungle’, ‘dripstone\_caves’;
- **试验 2:** ‘snowy\_plains’, ‘frozen\_river’, ‘dripstone\_caves’, ‘snowy\_taiga’, “海滩”;
- **试验 3:** ‘flower\_forest’, “草地”, ‘old\_growth\_birch\_forest’, ‘snowy\_slopes’, ‘frozen\_peaks’, “森林”, “河流”, “海滩”, “海洋”, ‘sunflower\_plains’, “平原”, ‘stony\_shore’.

ReAct [29] 在每次试验中穿越的地形为

- **试验 1:** “平原”, “沙漠”, “丛林”;
- **试验 2:** ‘snowy\_plains’, ‘snowy\_taiga’, ‘snowy\_slopes’;
- **试验 3:** ‘dark\_forest’, ‘dripstone\_caves’, “小树林”, ‘jagged\_peaks’.

Reflexion [30] 在每次试验中穿越的地形为

- **试验 1:** “平原”, ‘flower\_forest’;
- **试验 2:**
- **试验 3:** ‘old\_growth\_birch\_forest’, “河流”, “海洋”, “海滩”, “平原”。

AutoGPT [28] 在每次试验中穿越的地形是

- **试验 1:** “平原”, ‘dripstone\_caves’, “稀树草原”, “草地”;
- **试验 2:**
- **试验 3:** “平原”, ‘stony\_shore’, “森林”, “海洋”。

#### B.4.3 对未见任务的高效零样本泛化

另外两个任务对未见任务的零样本泛化结果如图 A.3 所示。与图 8 类似, VOYAGER 始终能解决所有任务, 而基线方法在 50 次提示迭代内无法解决任何任务。

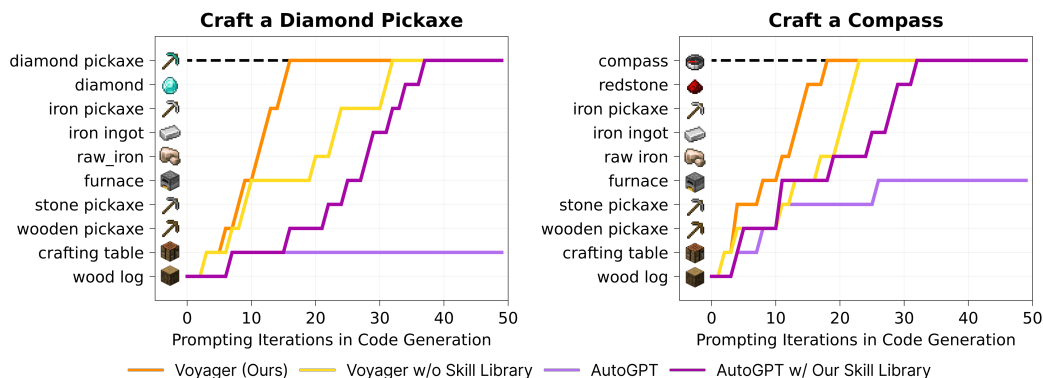


图 A.3: 对未见任务的零样本泛化。我们可视化了每种方法在另外两个任务上的中间进展。我们没有绘制 ReAct 和 Reflexion, 因为它们没有取得任何有意义的进展。

任务在 50 次提示迭代内。我们通过终身学习构建的技能库不仅提升了 VOYAGER 的性能, 还为 AutoGPT [28] 提供了助力。

#### B.4.4 准确的技能检索

我们对技能检索进行了评估 (共 309 个样本), 结果见表 A.4。前 5 准确率达到 96.5%, 表明我们的检索过程是可靠的 (注意, 我们在合成新技能的提示中包含了前 5 个相关技能)。

表 A.4: 技能检索准确率。前 1 准确率 前 2 准确率 前 3 准确率 前 4 准确率 前 5 准确率				
80.2 $\pm$ 3.0	89.3 $\pm$ 1.8	93.2 $\pm$ 0.7	95.2 $\pm$ 1.8	96.5 $\pm$ 0.3

#### B.4.5 对模型变化的鲁棒性

在主论文中, Voyager 的所有实验均使用 gpt-4-0314 进行。我们额外使用 gpt-4-0613 进行了新实验, 发现性能大致相同 (图 A.4)。这表明 Voyager 对模型变化具有鲁棒性。

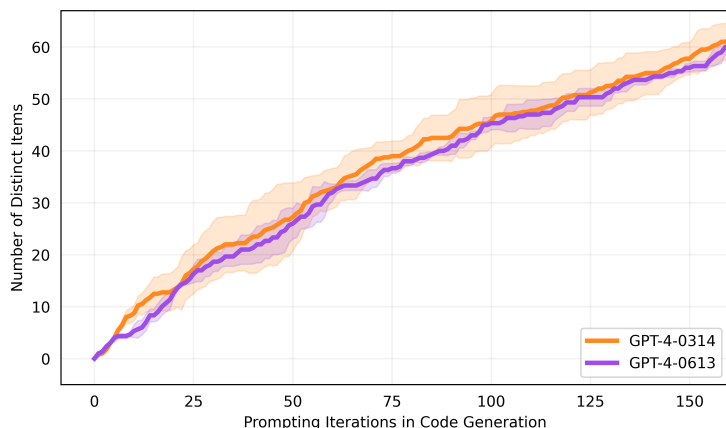


Figure A.4: VOYAGER’s performance with GPT-4-0314 and GPT-4-0613.